K-Nearest Neighbors, Decision Trees and Random Forests

Machine Learning 1 — Lecture 8

12th May 2026

Robert Peharz
Institute of Machine Learning and Neural Computation
Graz University of Technology

Algorithms for Supervised Learning

- so far, we discussed parametric models for supervised learning
  - linear models
  - linear models with non-linear features
  - neural networks
- each of these models was specified by a parameter vector $\boldsymbol{\theta}$
- we specified a training loss $\mathcal{L}_{train}(\boldsymbol{\theta})$ which we aimed to minimize w.r.t. $\boldsymbol{\theta}$
- there are, however, many other approaches to supervised learning

Algorithms for Supervised Learning

- today, we are going to discuss some classical algorithms:
  - **K-nearest neighbours (KNN)**
  - **decision trees**
  - **random forests**
- they are simple, robust, well-studied and often achieve strong performance out of the box
- standard implementations available, e.g. under `scikit-learn`
- "**no free lunch theorem:**" for any machine learning algorithm, there is some task where the algorithm does not perform best when provided with finitely many data
- thus, getting best performance requires trying different learning algorithms

K-Nearest Neighbours

- assume a training set $\mathcal{D} = \{(\boldsymbol{x}^{(1)}, y^{(1)}), \ldots, (\boldsymbol{x}^{(N)}, y^{(N)})\}$
- no training required: just dump the dataset in your memory
- assume a test sample $\boldsymbol{x}^*$ for which we don't know target y^*
- let i be the index of the **nearest neighbour** of $\boldsymbol{x}^*$, i.e.

$$i = \arg\min_i \|\boldsymbol{x}^* - \boldsymbol{x}^{(i)}\|$$

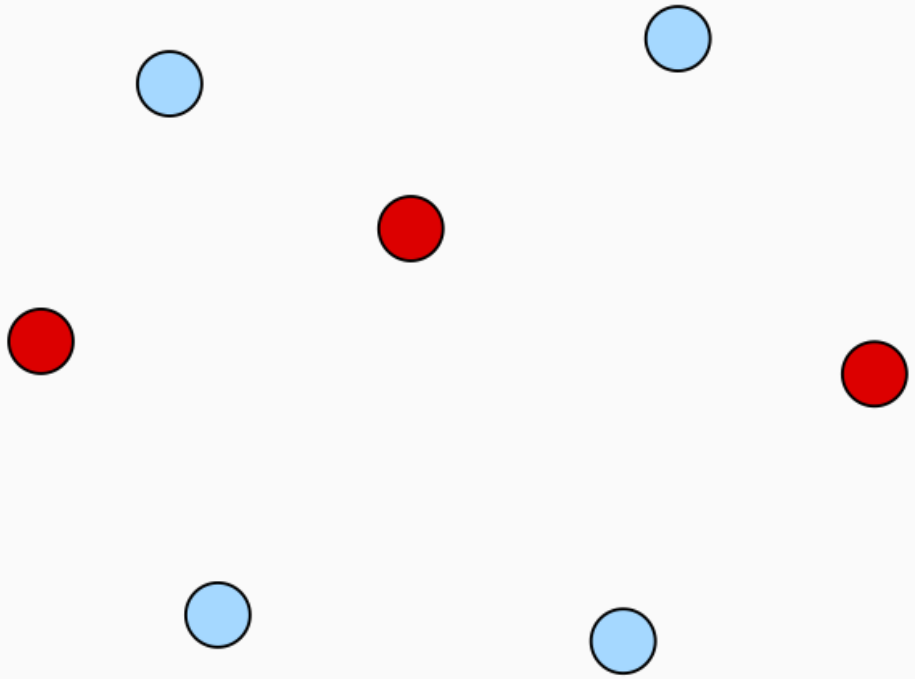
  for some norm $\|\cdot\|$
- the **nearest neighbour** predictor returns the target of the nearest neighbour

$$f_{NN}(\boldsymbol{x}^*) = y^{(i)}$$

- note that this works for both classification and regression

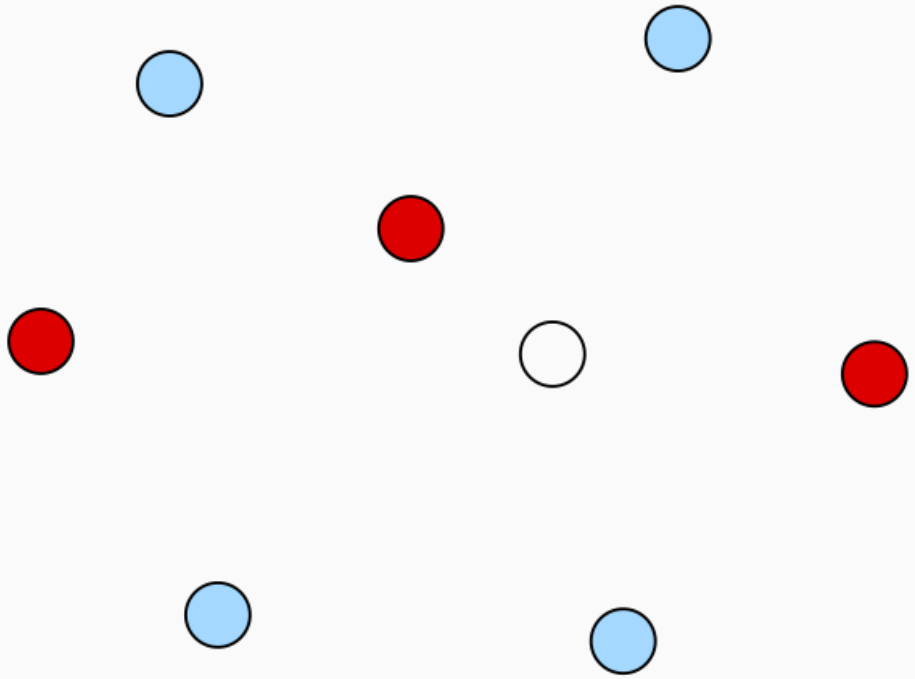
Nearest Neighbour Classifier

Example



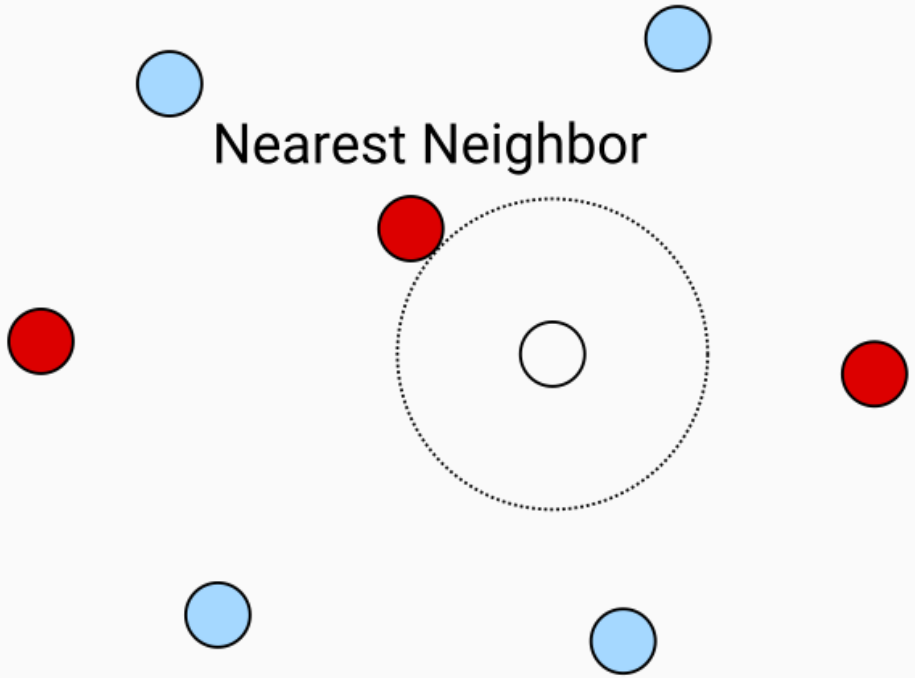
Nearest Neighbour Classifier

Example



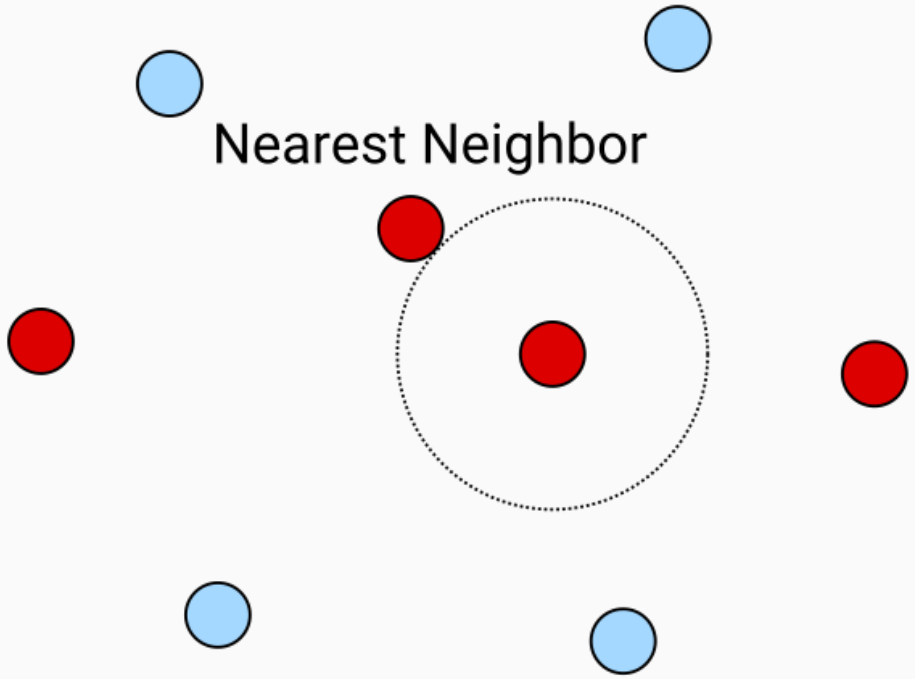
Nearest Neighbour Classifier

Example



Nearest Neighbour Classifier

Example

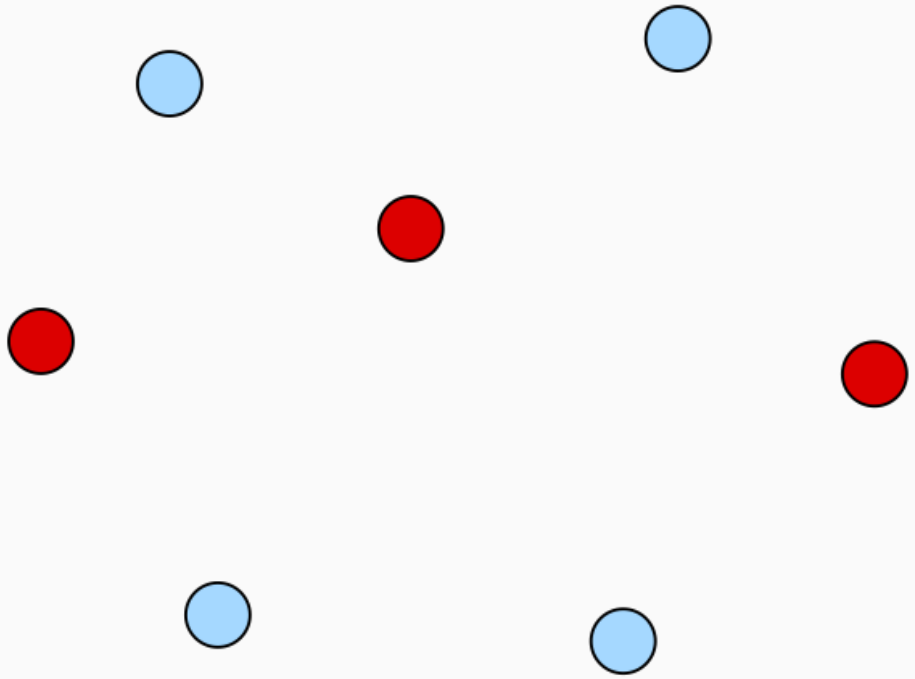


- why should we restrict to only one neighbour?
- instead, we can combine the targets of the K **nearest neighbours**
- sort training points according to distance to $\boldsymbol{x}^*$:

$$\|\boldsymbol{x}^{(i_1)} - \boldsymbol{x}^*\| \leq \|\boldsymbol{x}^{(i_2)} - \boldsymbol{x}^*\| \leq \cdots \leq \|\boldsymbol{x}^{(i_K)} - \boldsymbol{x}^*\| \leq \cdots \leq \|\boldsymbol{x}^{(i_N)} - \boldsymbol{x}^*\|$$

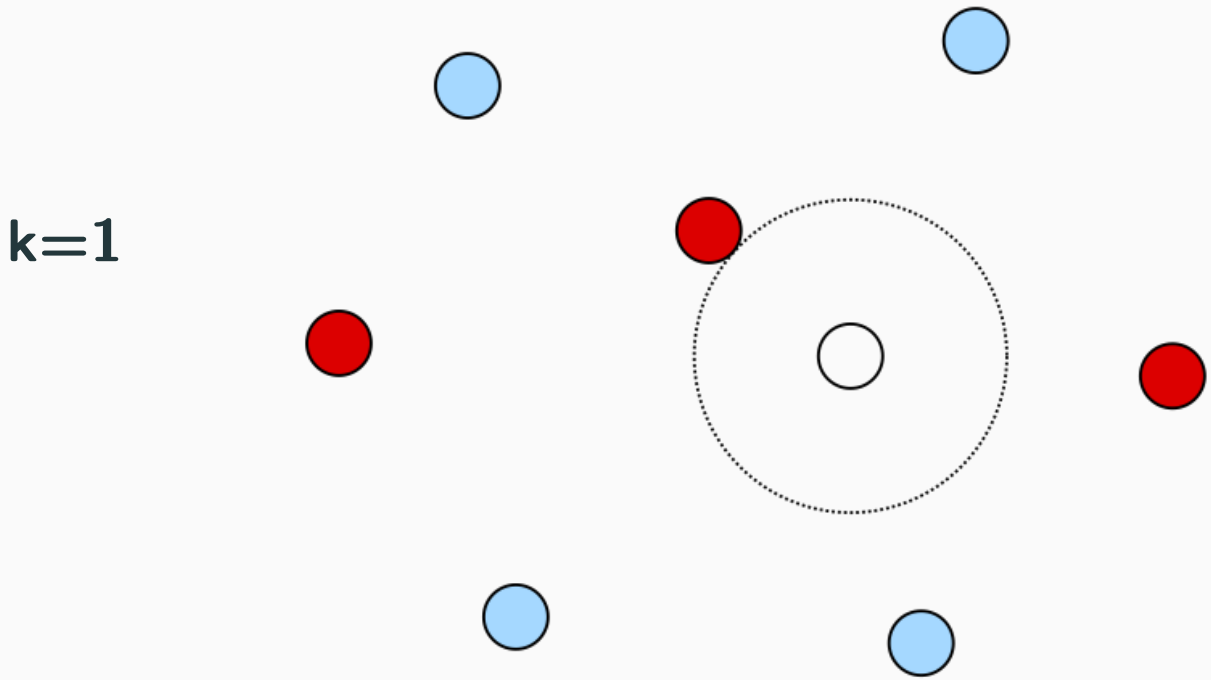
- **classification:** label test sample according to **majority vote** of K nearest neighbours. Break ties randomly.
- **regression:** average targets of K nearest neighbours

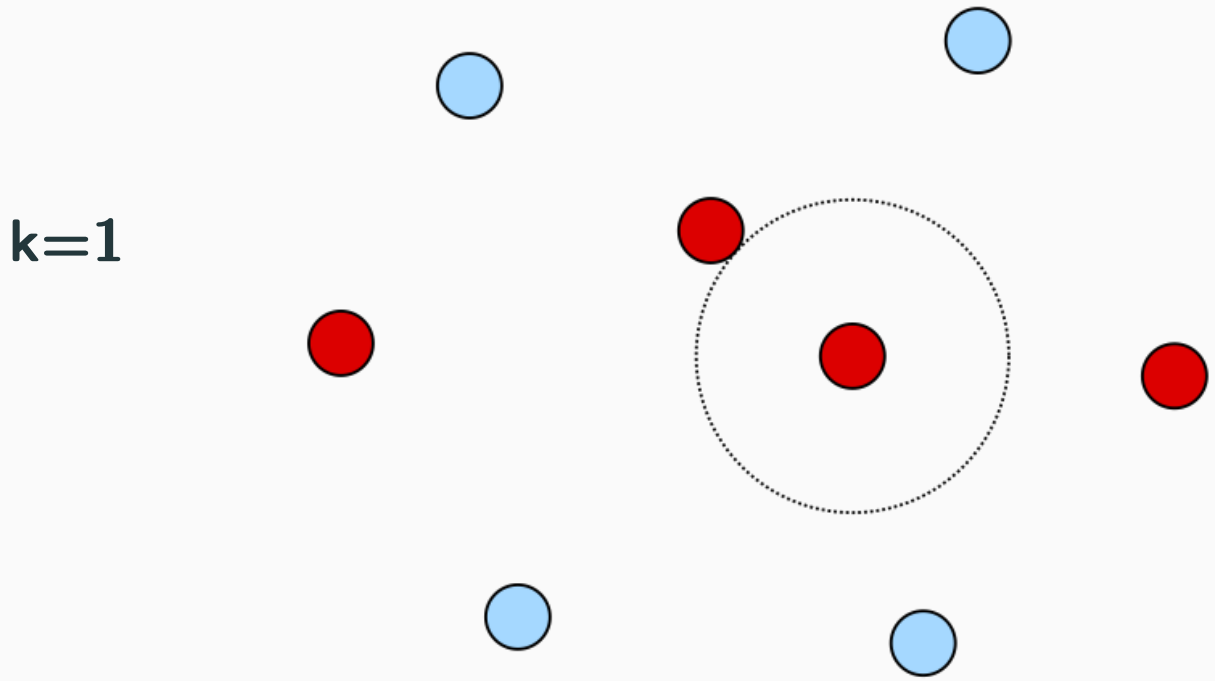
$$\hat{y} = \frac{1}{K} \sum_{k=1}^{K} y^{(i_k)}$$



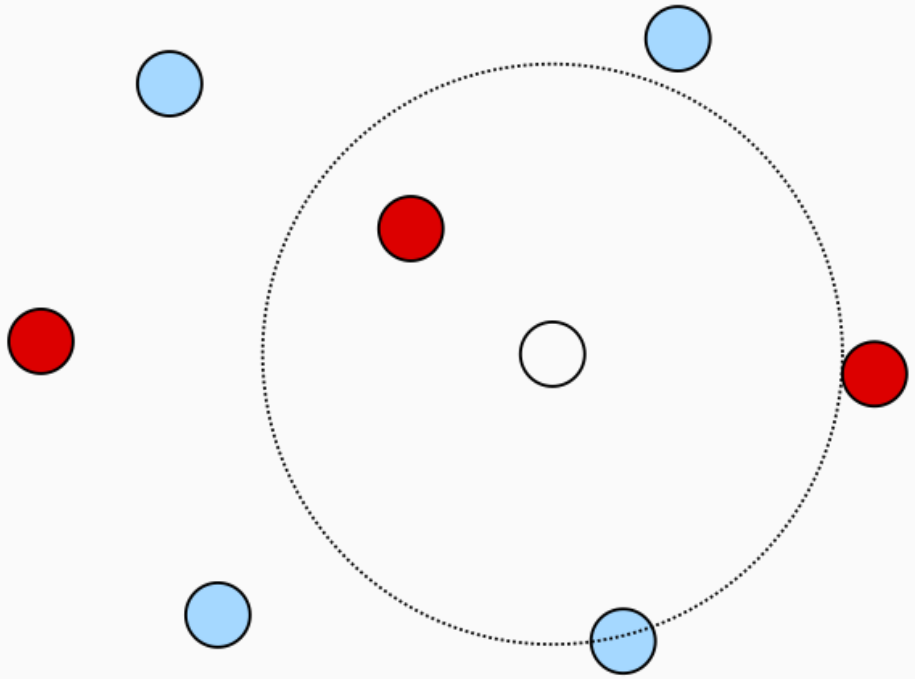
KNN Classification

Example




k=1

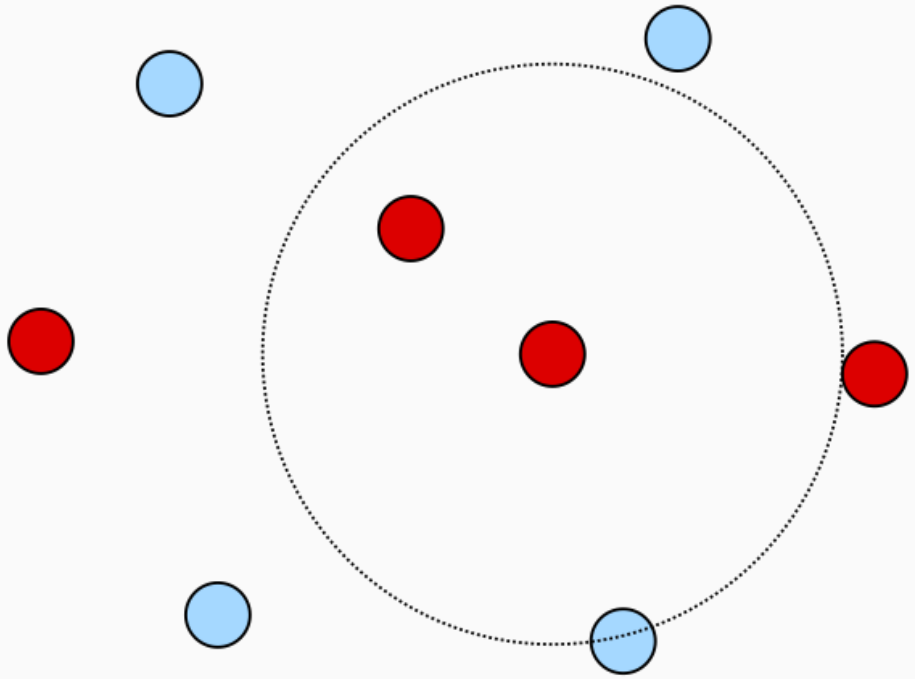
k=3



KNN Classification

Example

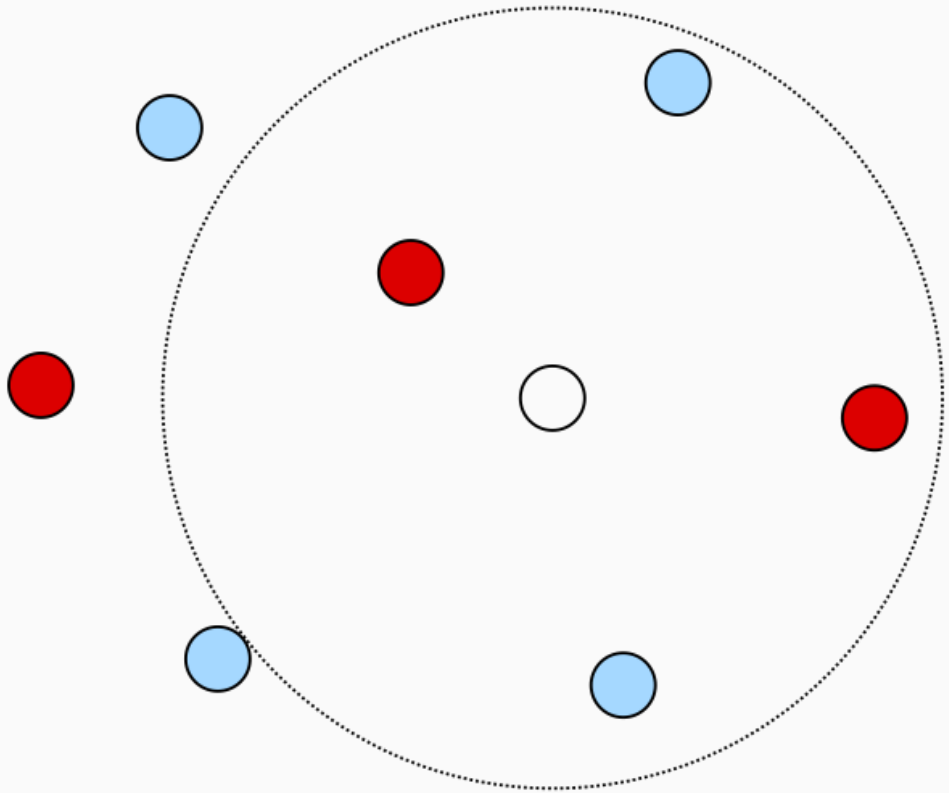
k=3



KNN Classification

Example

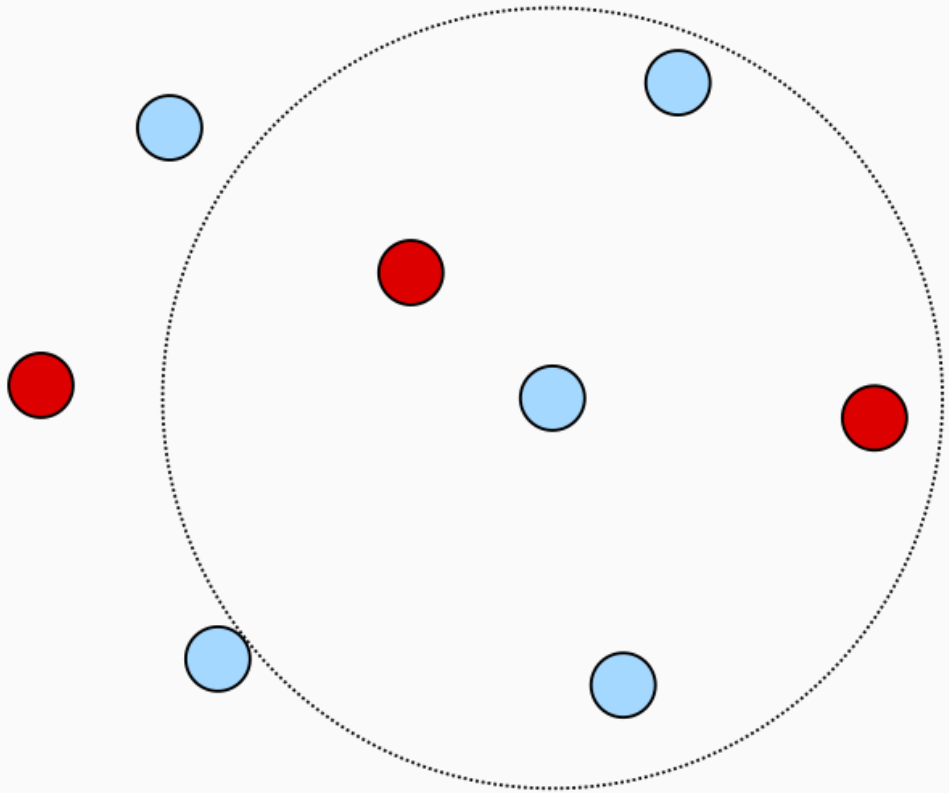
k=5



KNN Classification

Example

k=5



Bayes Optimal Classifier

Definition

- assume a classification setting (y discrete)
- assume we would know the **true data-generating** distribution $p_{\text{true}}(\boldsymbol{x}, y)$ (**Recall Lecture 6**)
- then we could compute $p_{\text{true}}(y \mid \boldsymbol{x}) = \frac{p_{\text{true}}(\boldsymbol{x},y)}{p_{\text{true}}(\boldsymbol{x})} = \frac{p_{\text{true}}(\boldsymbol{x},y)}{\sum_y p_{\text{true}}(\boldsymbol{x},y)}$
- then, a natural classification rule would be:

$$\hat{y} = \arg\max_y \, p_{\text{true}}(y \mid \boldsymbol{x})$$

- this rule is known as the **Bayes optimal classifier**
- it suffers the least **true classification error**, i.e.

$$\mathbb{E}_{\boldsymbol{x},y\sim p_{\text{true}}} \left[\mathbb{1}(\hat{y} \neq y)\right]$$

  is minimal among all possible classifiers.

Consistency of KNN

Theorem

KNN is **simple**, but **theoretically sound!**

In particular, if

- number of training points $N \to \infty$
- number of neighbours $K \to \infty$
- N grows quicker than K, i.e. $\frac{K}{N} \to 0$, e.g. when $K = \sqrt{N}$

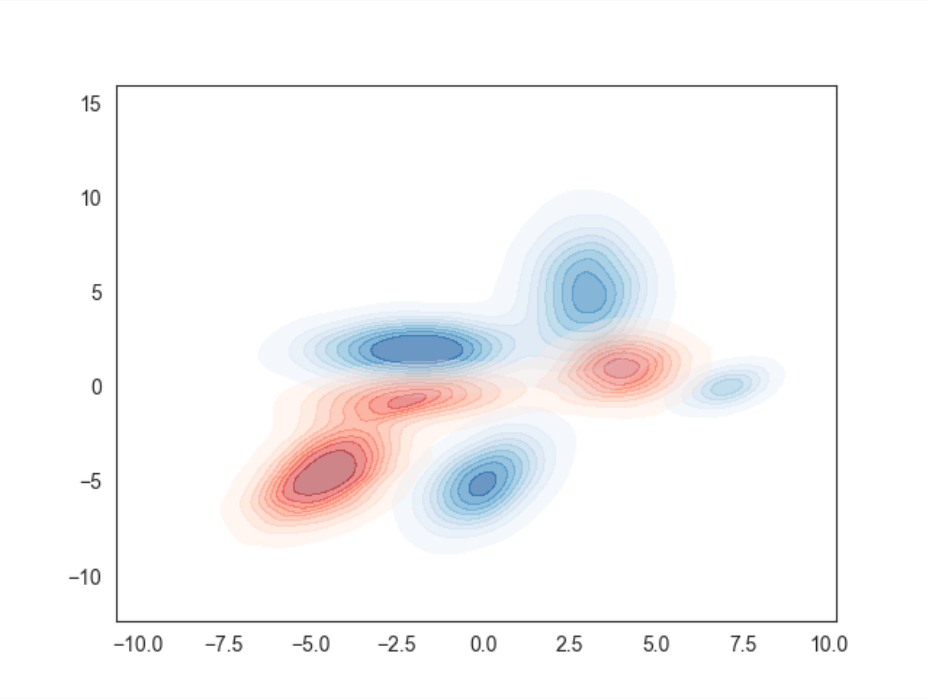
then KNN converges towards the **Bayes optimal classifier**.

A classifier with this property is called **consistent**.

Furthermore, for regression, KNN converges to the **optimal regression function** $\mathbb{E}_{p_{\text{true}}(y \mid \boldsymbol{x})}[y]$.

We can demonstrate Bayes consistency of KNN on a toy dataset.

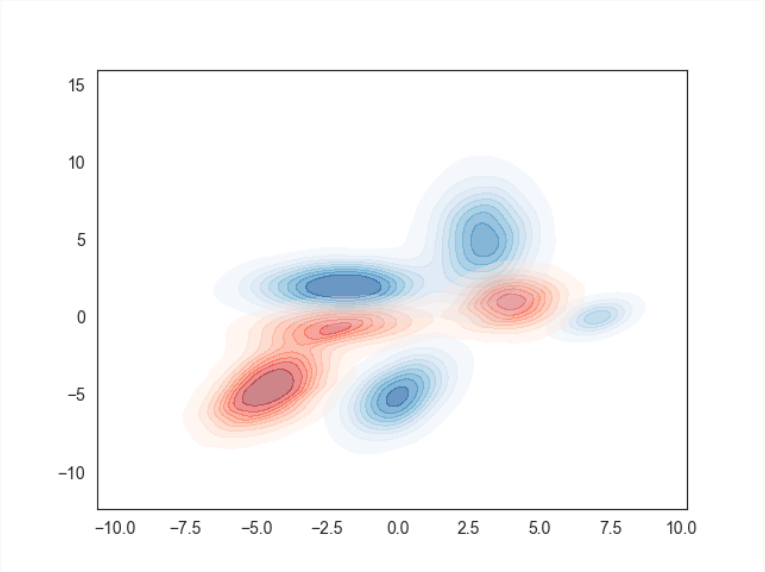
Assume a binary classification problem and assume the following true data distribution $p_{\text{true}}(\boldsymbol{x}, y)$ (red and blue for the two classes):



This is a **Gaussian mixture**, to be discussed in a future lecture.

Bayes Consistency of KNN cont'd

Example



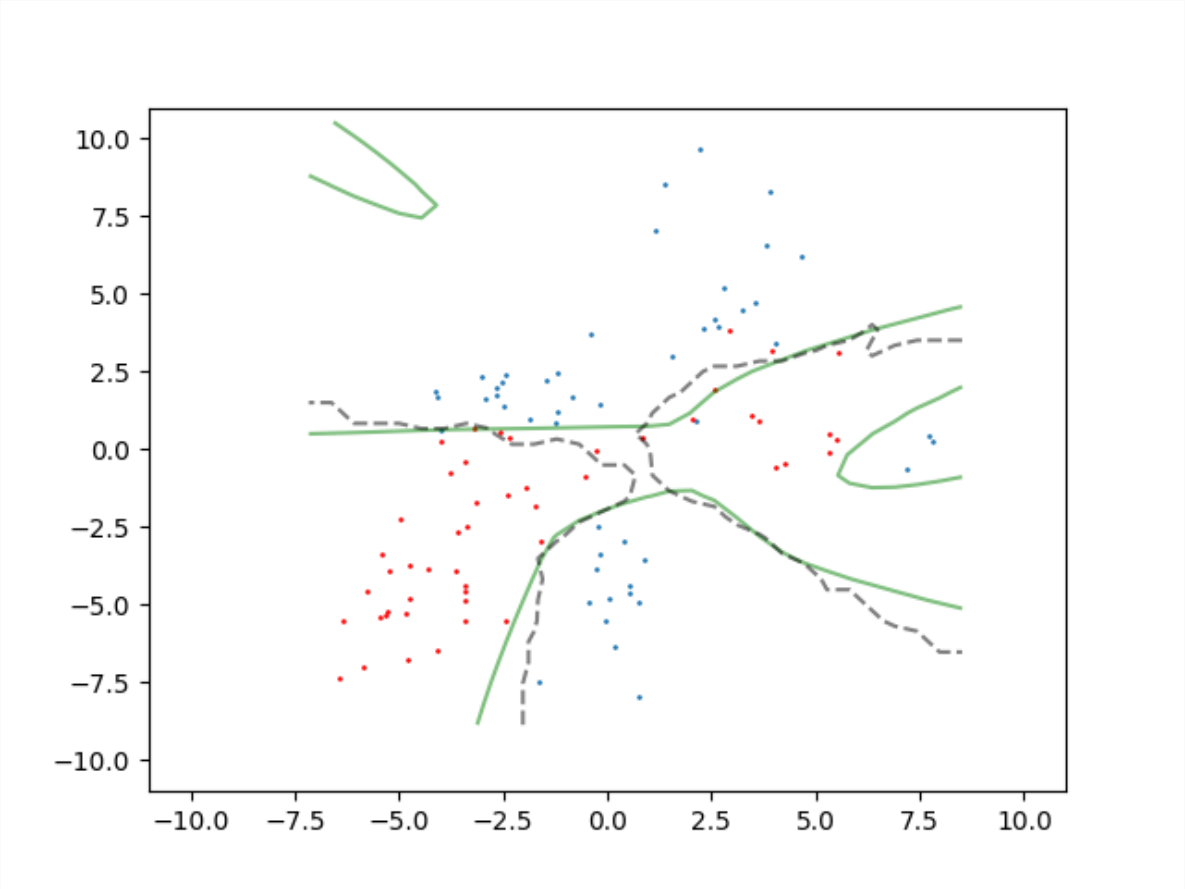
- in this **toy example** the **density is known**, hence we can actually compute the Bayes optimal classifier:

$$\hat{y} = \arg\max_{y} \; p_{\text{true}}(y \mid \boldsymbol{x})$$

- we can also **easily sample** and simulate training data
- test performance is computed on 1 million independent test samples

Bayes Consistency of KNN cont'd

Example



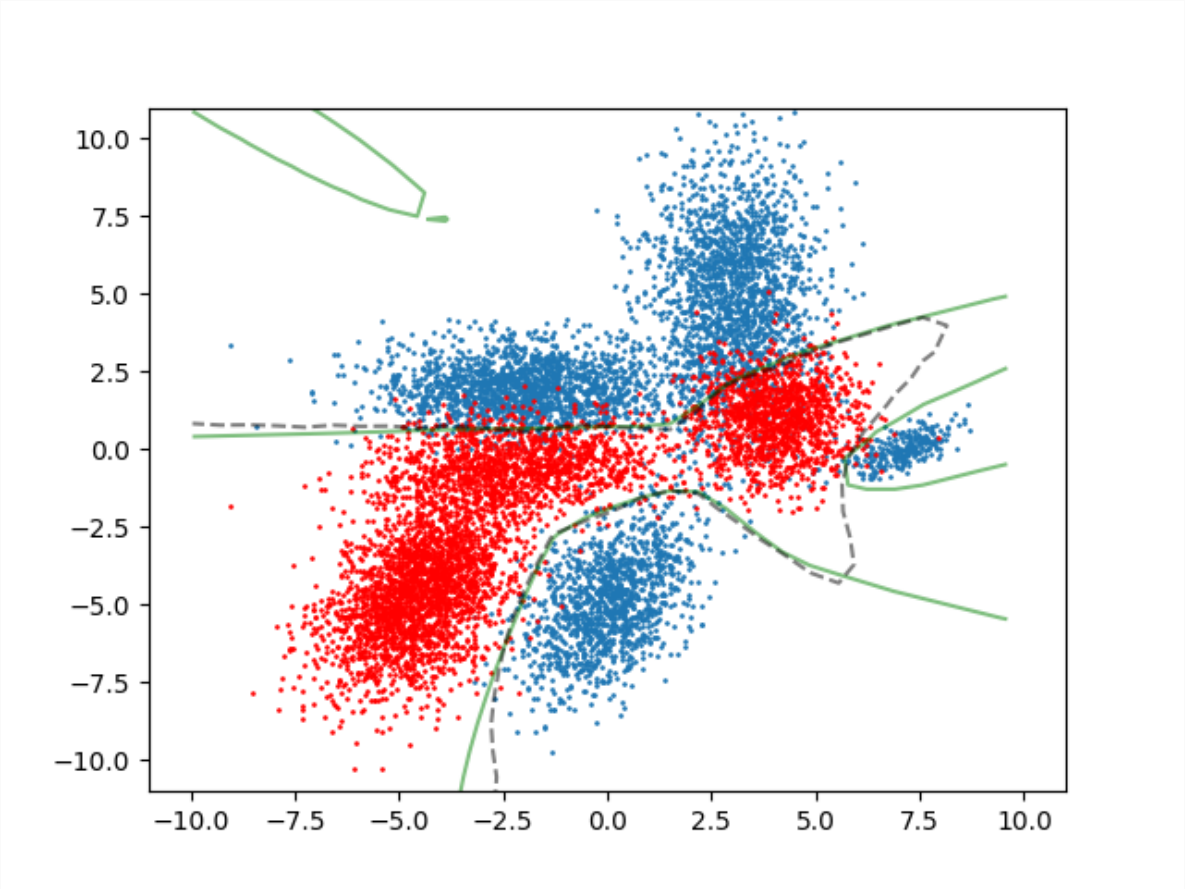
Green line: Bayes optimal classifier (94.67% test accuracy)

Dashed: KNN $N_{train} = 100$ and $K = 9$ (90.26% test accuracy)

(accuracies computed on a 1 million test samples.)

Bayes Consistency of KNN cont'd

Example



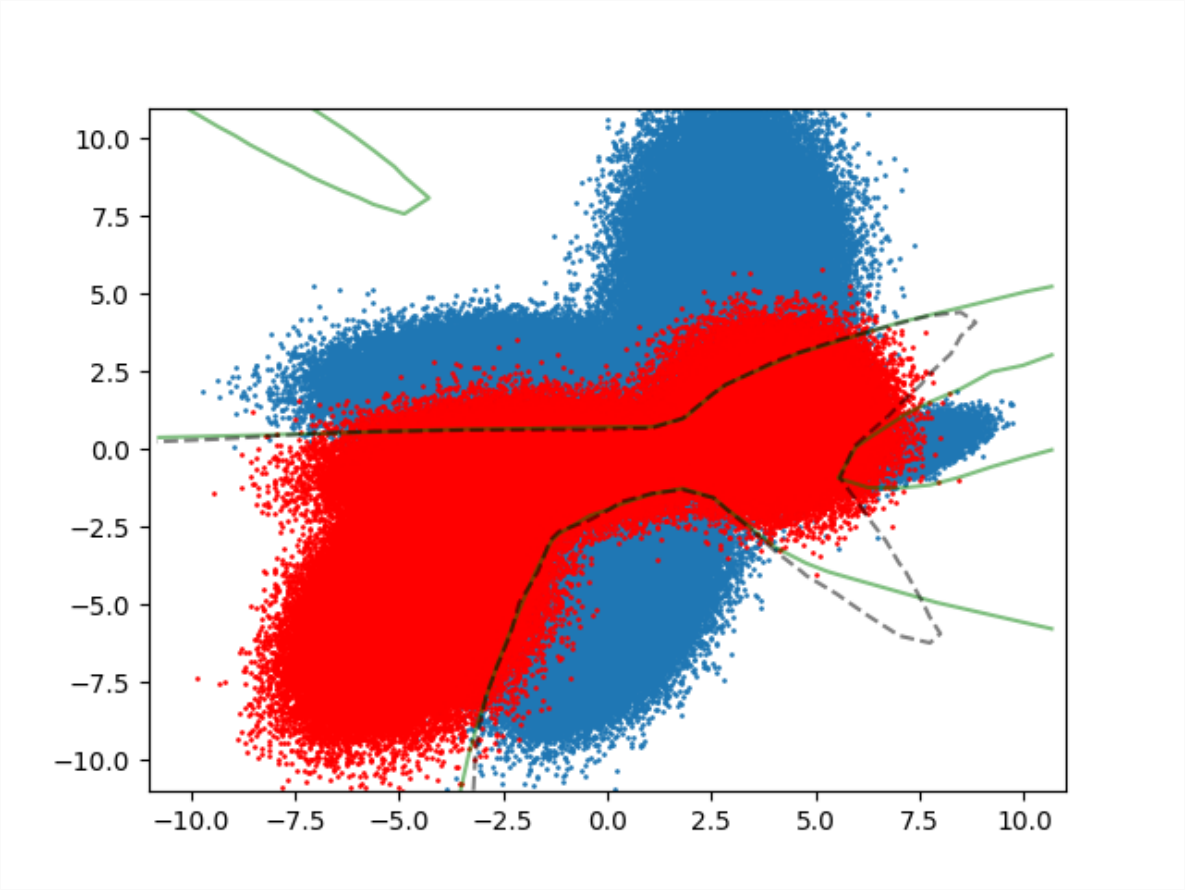
Green line: Bayes optimal classifier (94.67% test accuracy)

Dashed: KNN $N_{train} = 10k$ and $K = 99$ (94.56% test accuracy)

(accuracies computed on a 1 million test samples.)

Bayes Consistency of KNN cont'd

Example



Green line: Bayes optimal classifier (94.67% test accuracy)

Dashed: KNN $N_{train} = 1M$ and $K = 999$ (94.65% test accuracy)

(accuracies computed on a 1 million test samples.)

- simple but consistent predictor
- training is very easy: just store the data
- testing is $\mathcal{O}(N)$
- **k-d trees** organize the data in hierarchical way, then testing can be done on $\mathcal{O}(\log N)$

Decision Trees



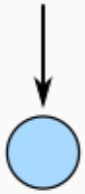


features **x**: weight, diameter, color, acid, ...

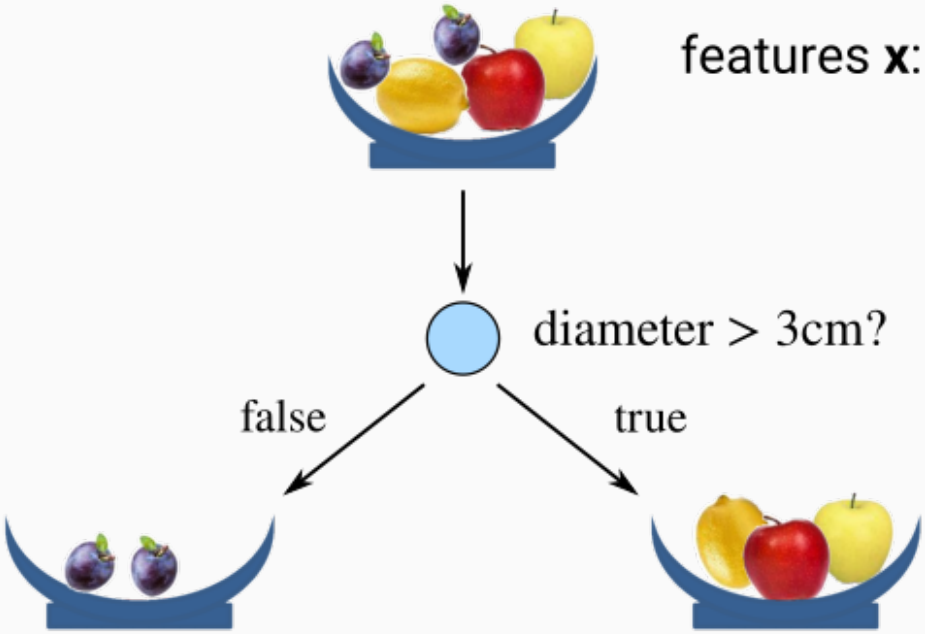
Example



features **x**: weight, diameter, color, acid, ...

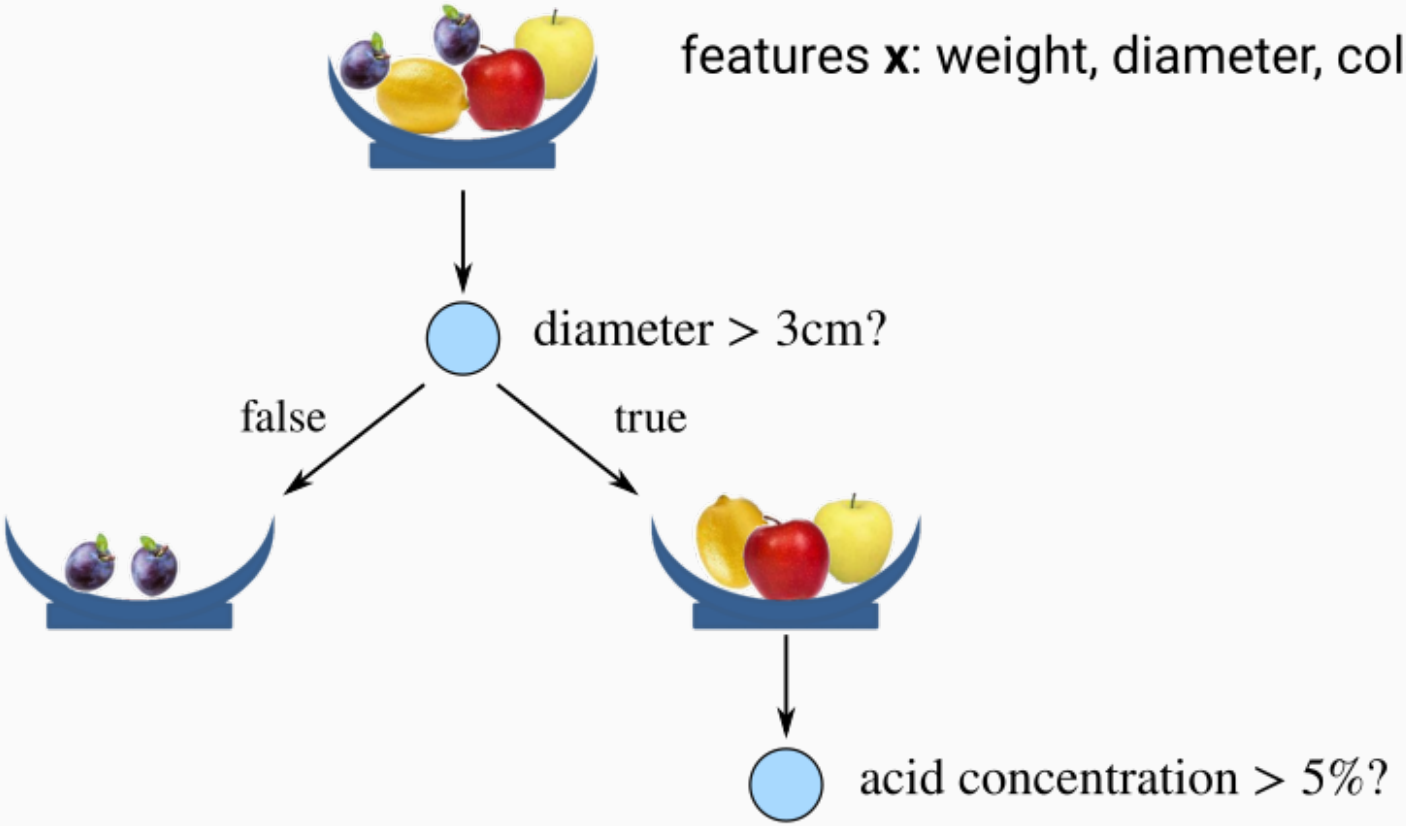


Example

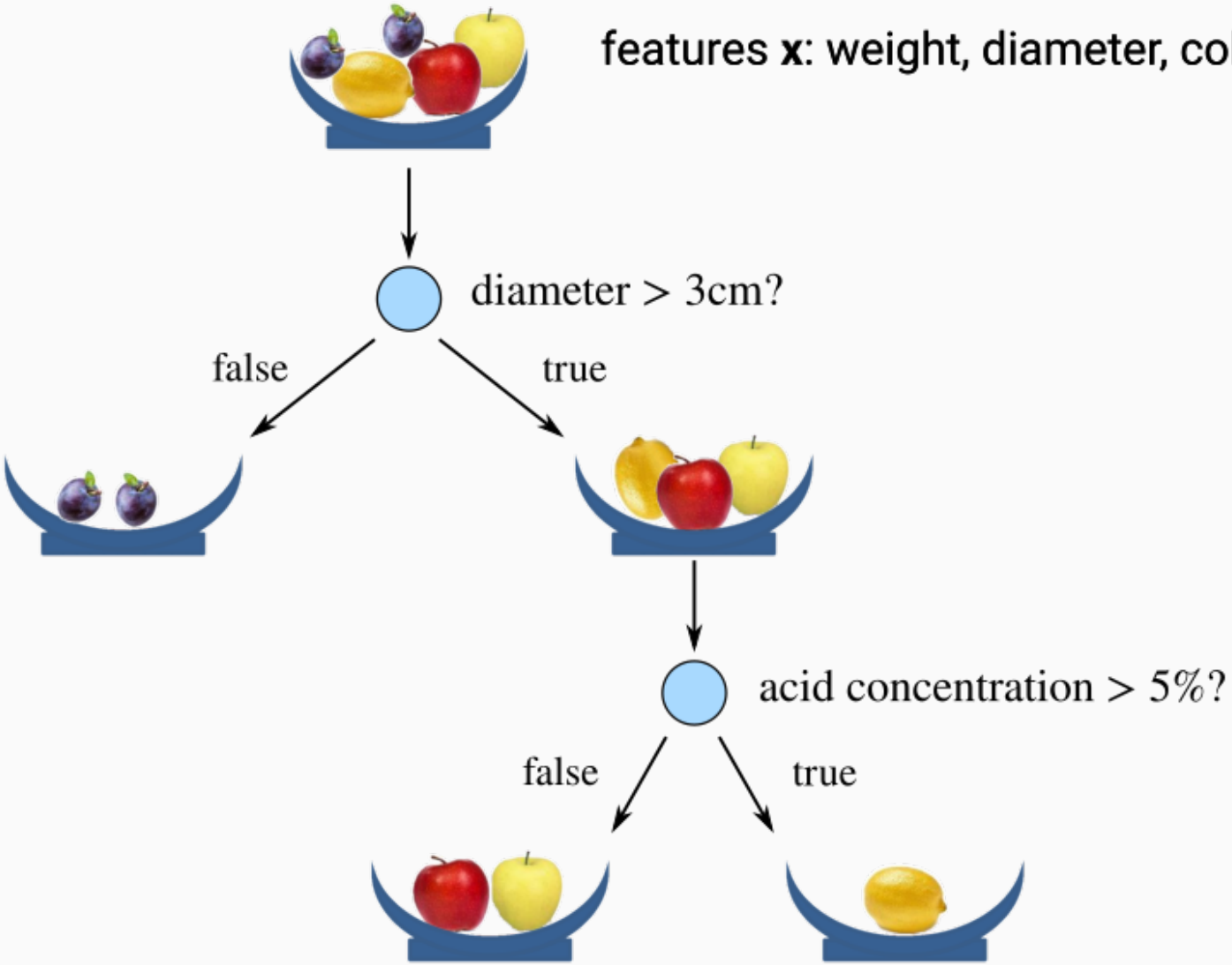


features **x**: weight, diameter, color, acid, ...

Example


features **x**: weight, diameter, color, acid, ...
diameter > 3cm?
false
true
acid concentration > 5%?

Example


features **x**: weight, diameter, color, acid, ...
diameter > 3cm?
false
true
acid concentration > 5%?
false
true

Given a set of features $X_1, X_2, \ldots, X_D$ (discrete or continuous) and a class variable Y, a **decision tree** is a **directed binary tree** with two types of nodes:

- **decision nodes** (internal nodes)
  decision nodes are associated with a feature X_i and a boolean function (**decision**, **test**)

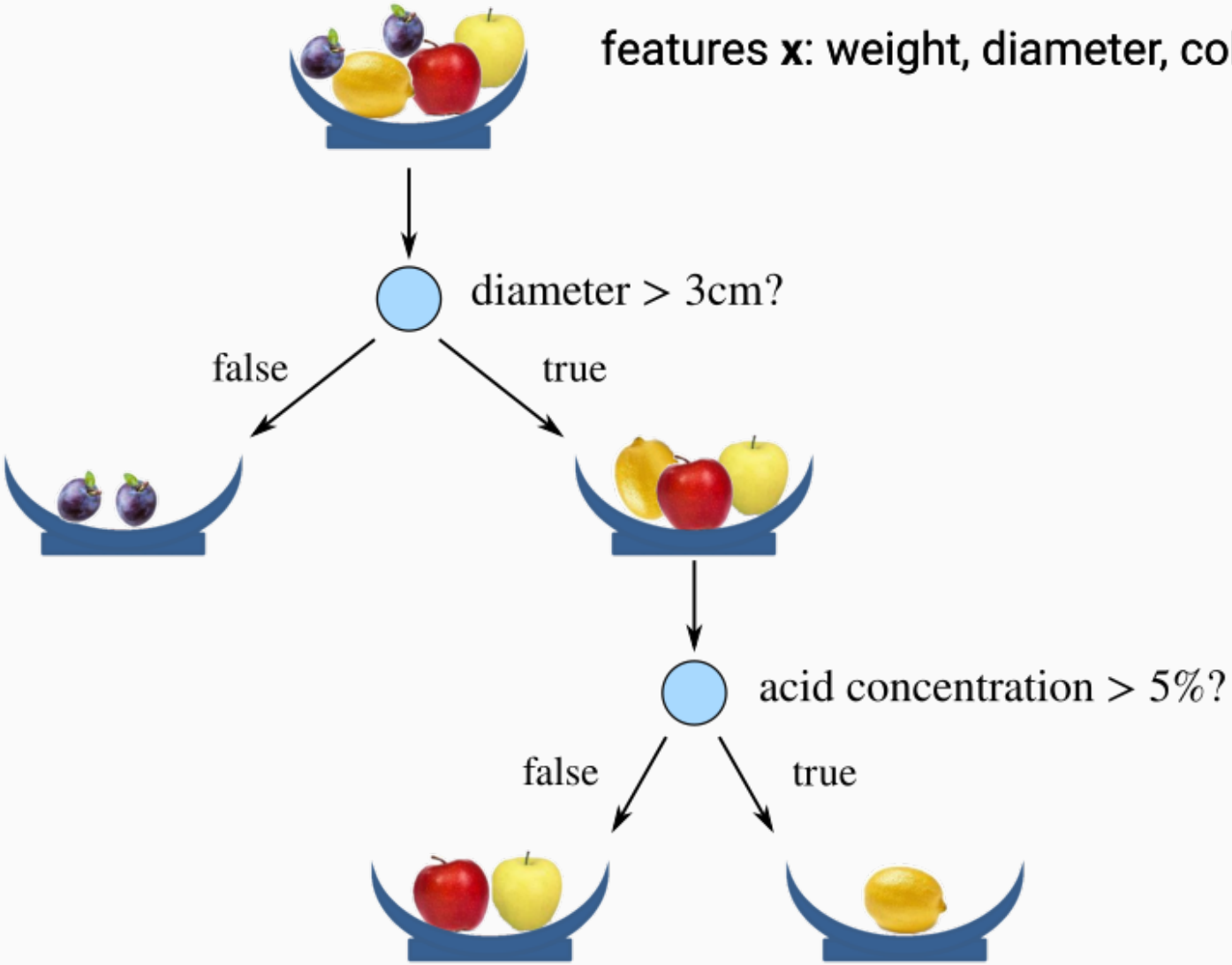
  $$f : X_i \mapsto \{true, false\}$$

  decision nodes have 2 children labeled *true* and *false*, which are selected by the output of f.
- **prediction nodes** (leaves)
  prediction nodes are associated with either
  - a fixed label y
  - a distribution over labels $p(y)$

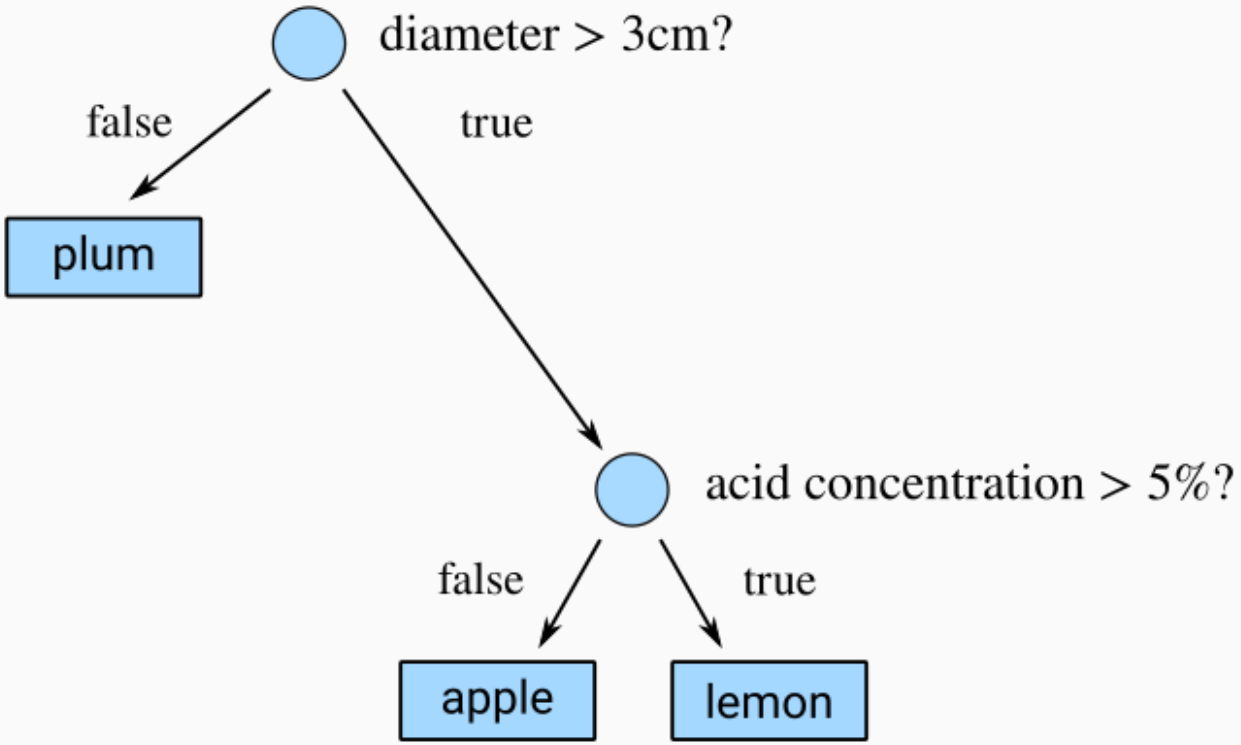
Classification using a Decision Tree

- parse decisions from root to leaf
- a leaf is selected, if **all** tests on the path from root to leaf are true
- selected leaf is guaranteed to be unique, since decision nodes select exactly one child
- use the predictor in the leaf for classification:
  - a fixed label y
  - a distribution over labels $p(y)$
- easily generalized to non-binary decision nodes
- decision trees can also be used for regression (not discussed)

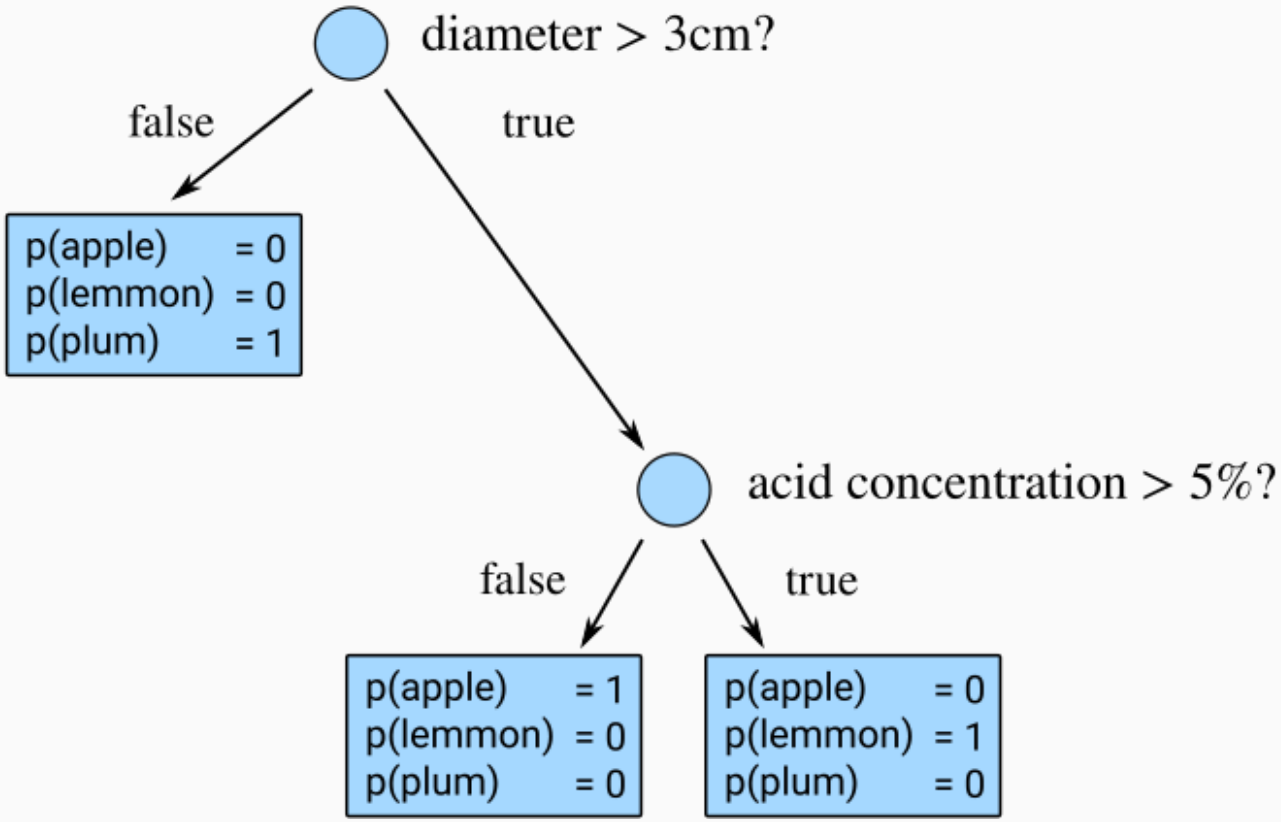
Example


features **x**: weight, diameter, color, acid, ...
diameter > 3cm?
false
true
acid concentration > 5%?
false
true

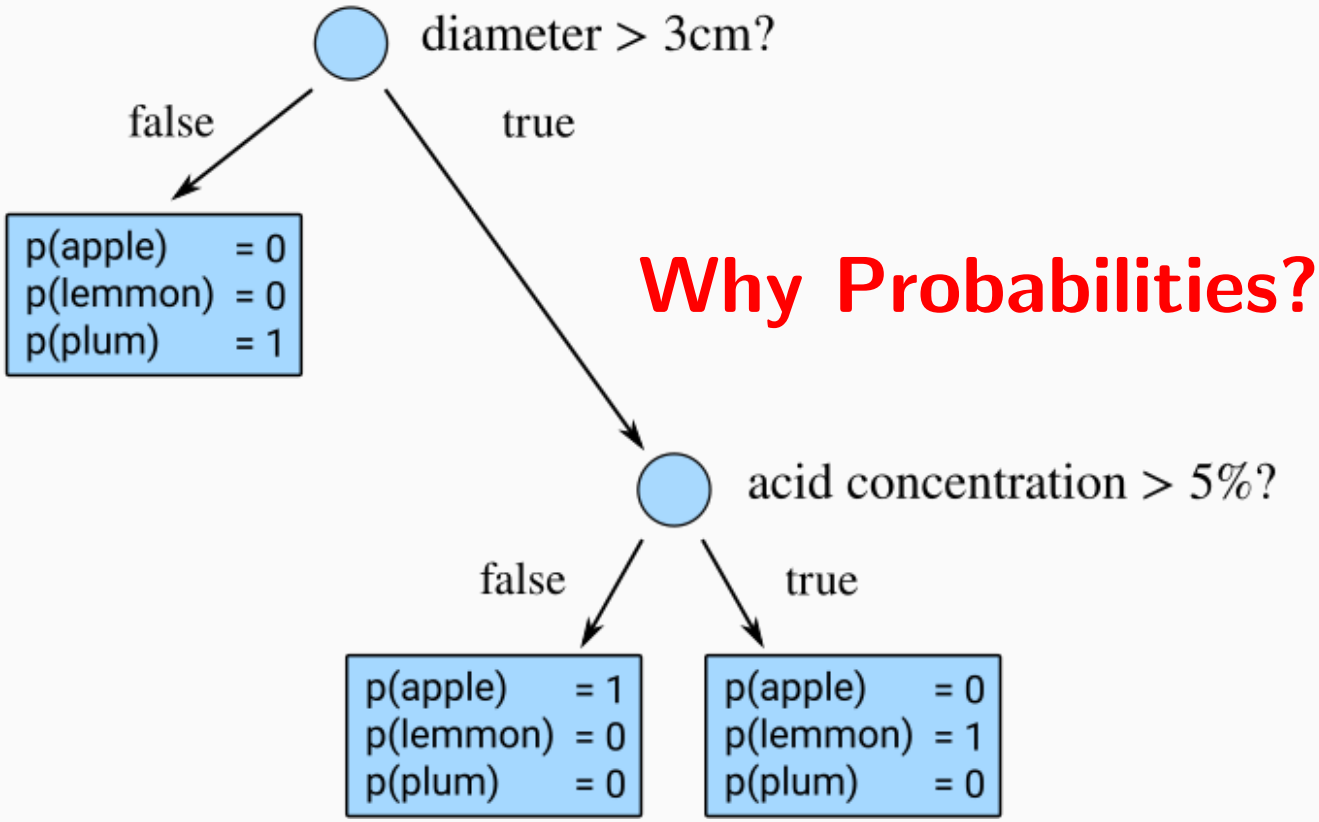
features **x**: weight, diameter, color, acid, ...



features **x**: weight, diameter, color, acid, ...

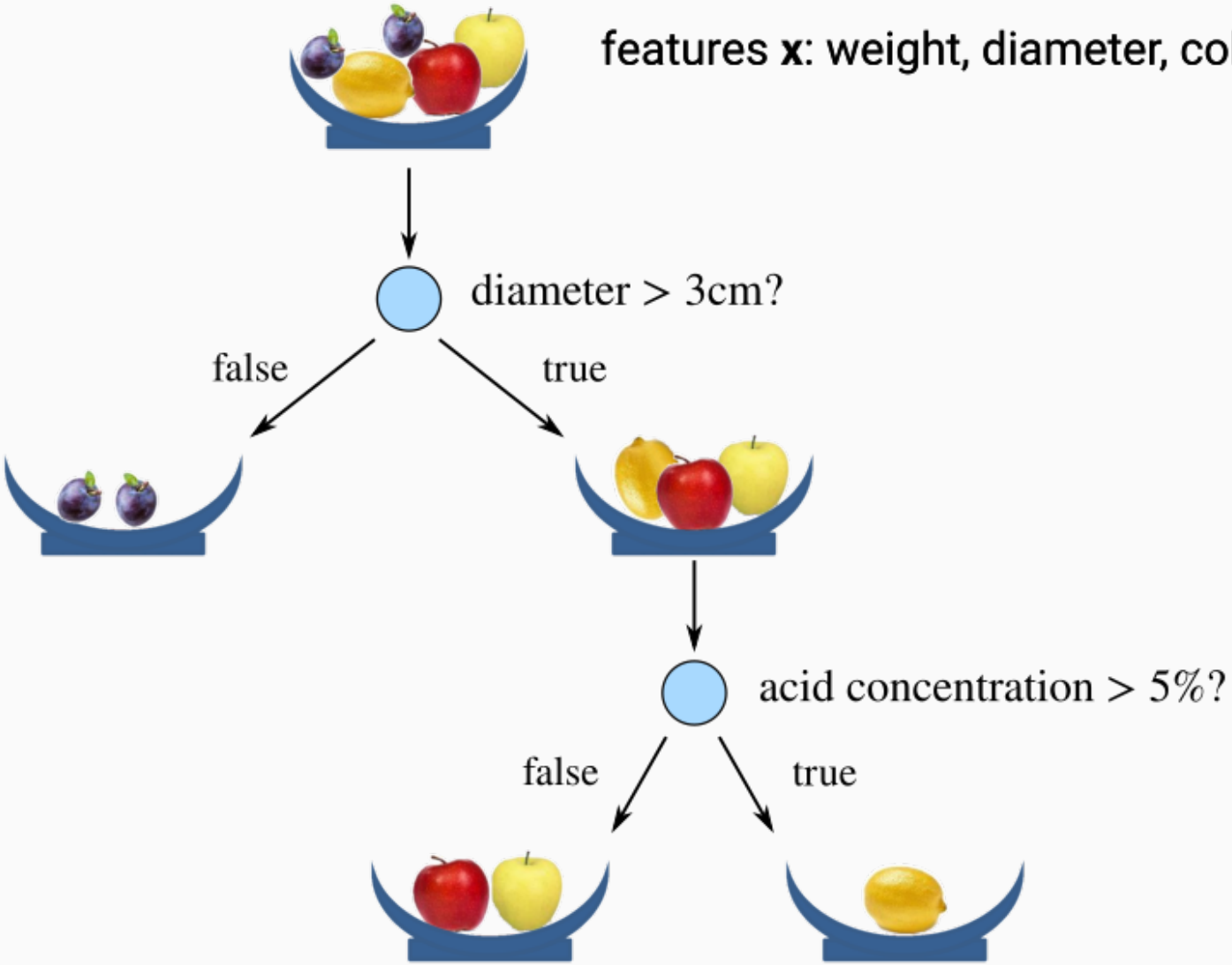


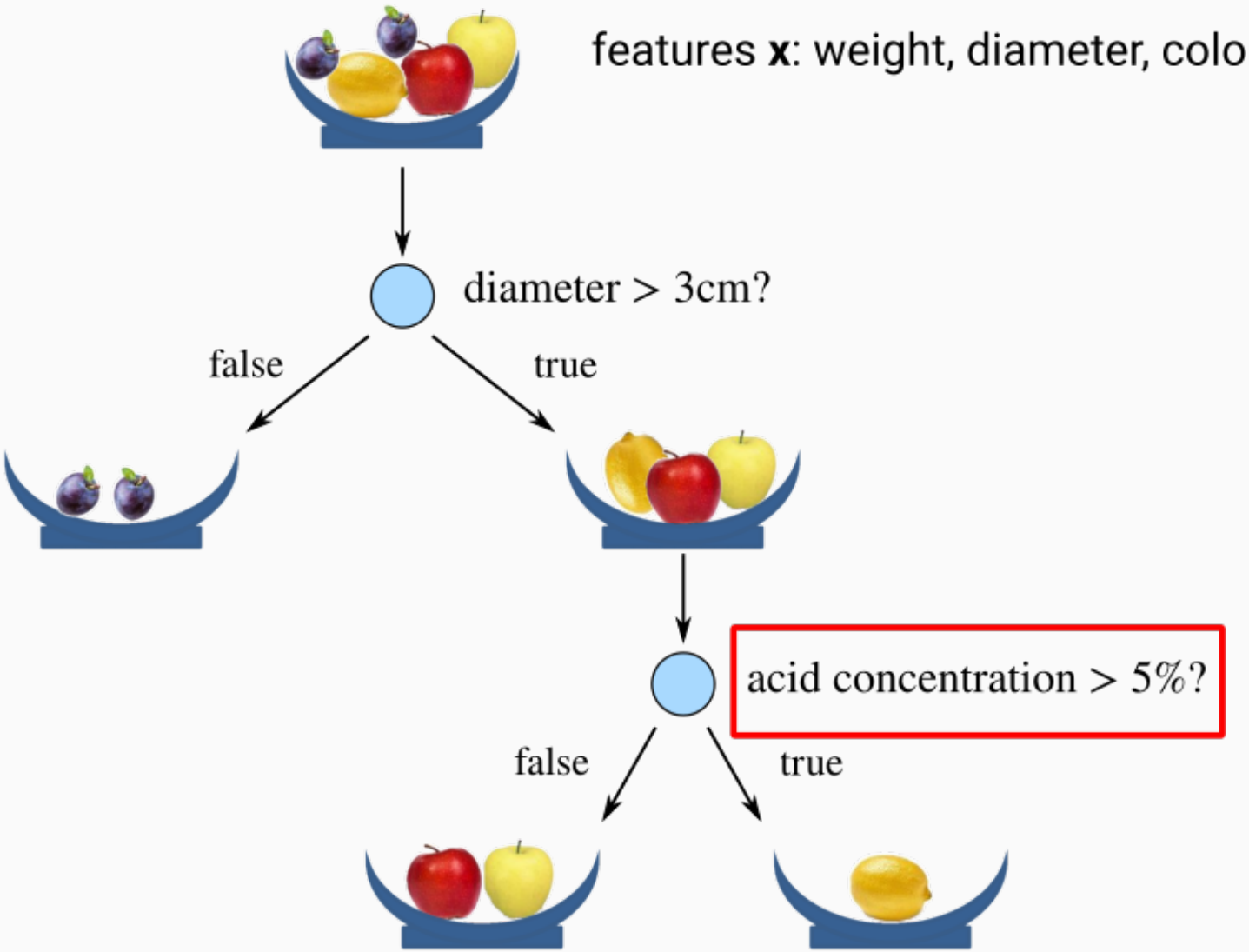
features **x**: weight, diameter, color, acid, ...

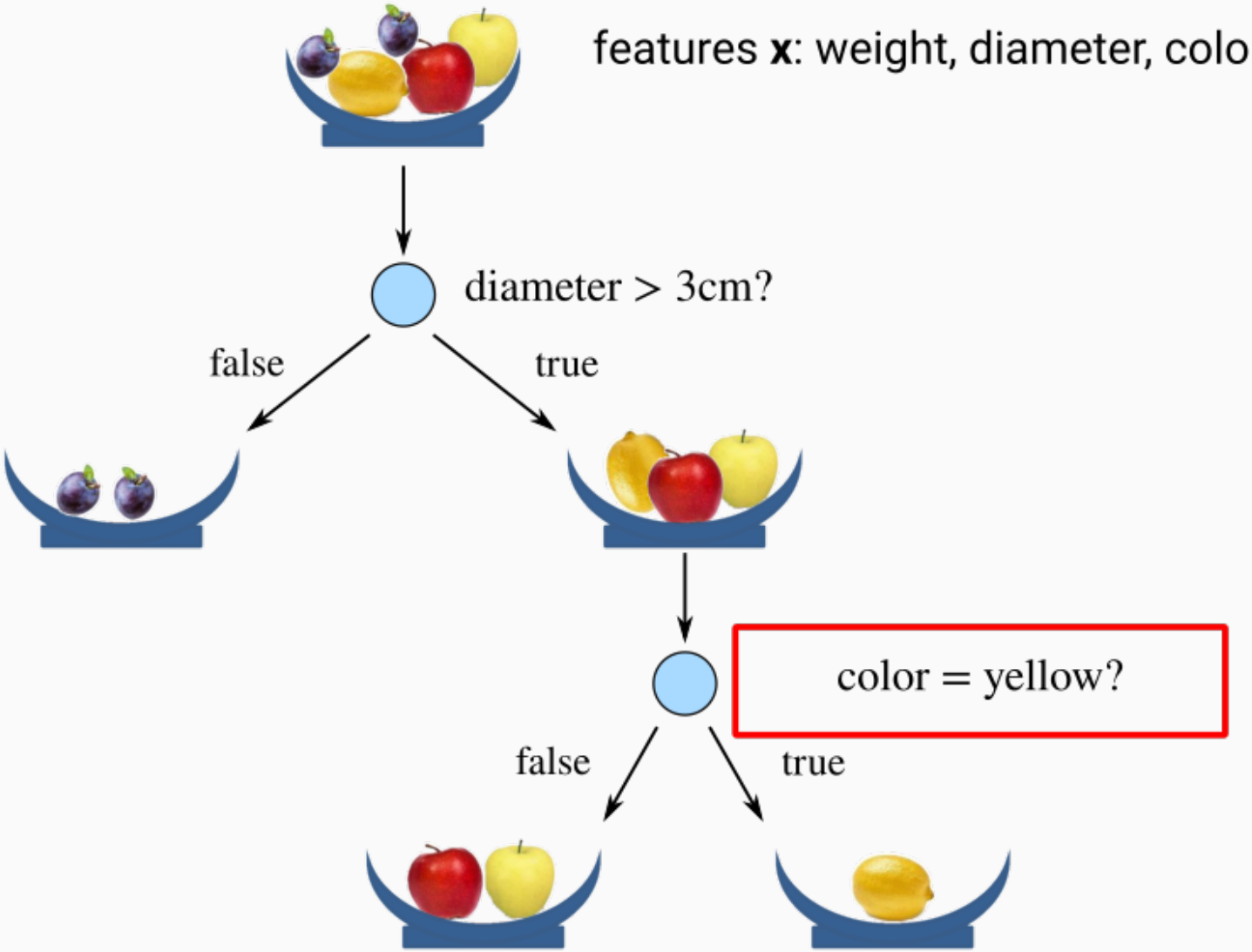


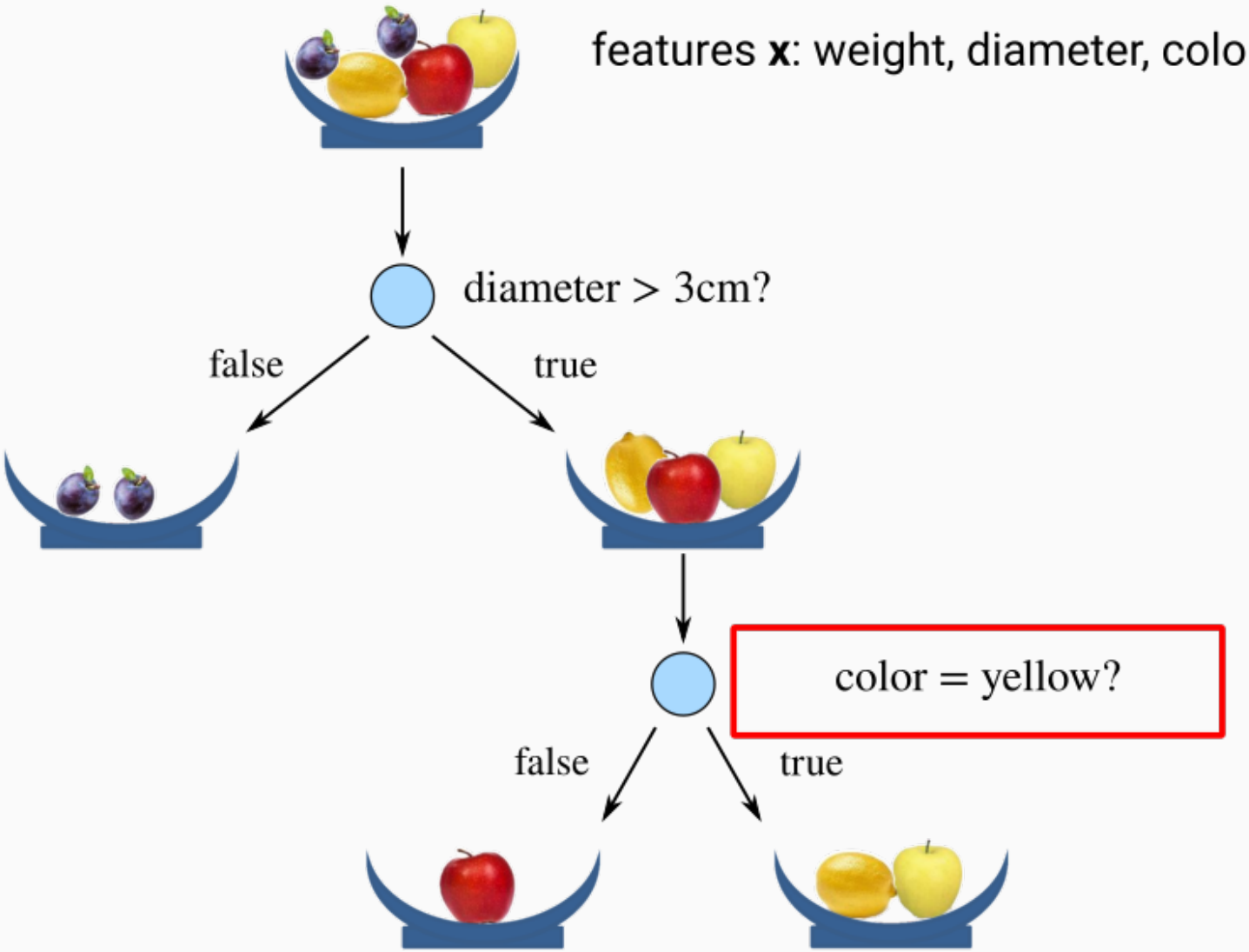
Why Probabilities?

Example

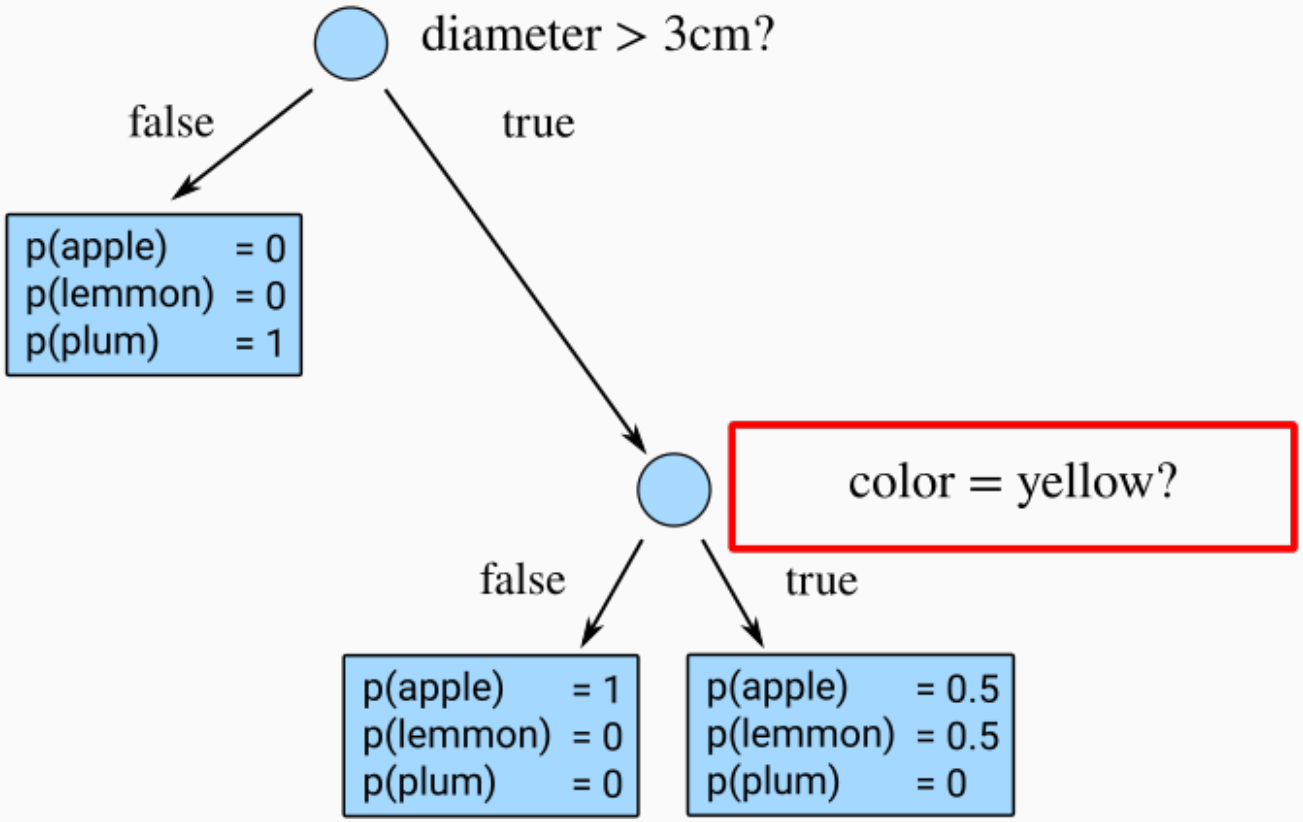

features **x**: weight, diameter, color, acid, ...
diameter > 3cm?
false
true
acid concentration > 5%?
false
true


features **x**: weight, diameter, color, acid, ...
diameter > 3cm?
false
true
acid concentration > 5%?
false
true

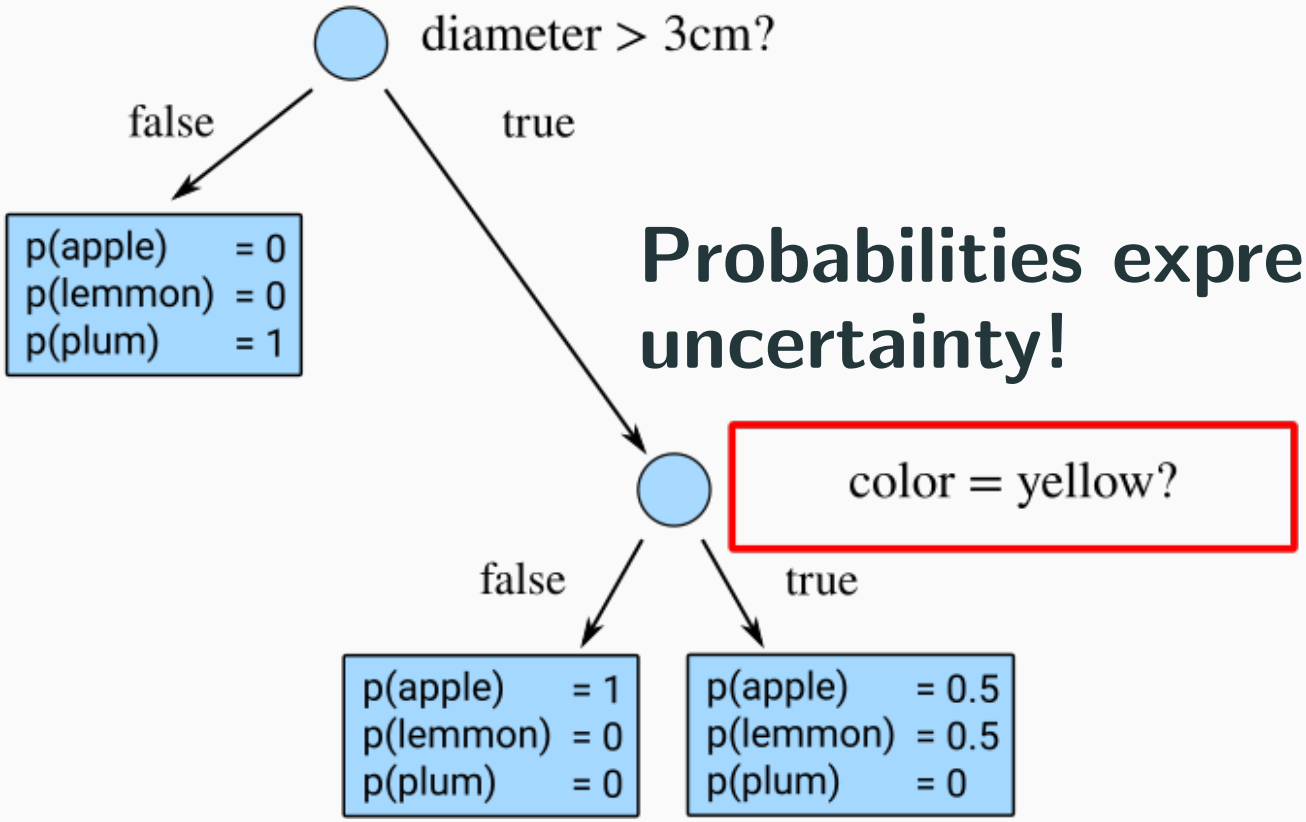

features **x**: weight, diameter, color, acid, ...
diameter > 3cm?
false
true
color = yellow?
false
true


features **x**: weight, diameter, color, acid, ...
diameter > 3cm?
false
true
color = yellow?
false
true

features **x**: weight, diameter, color, acid, ...



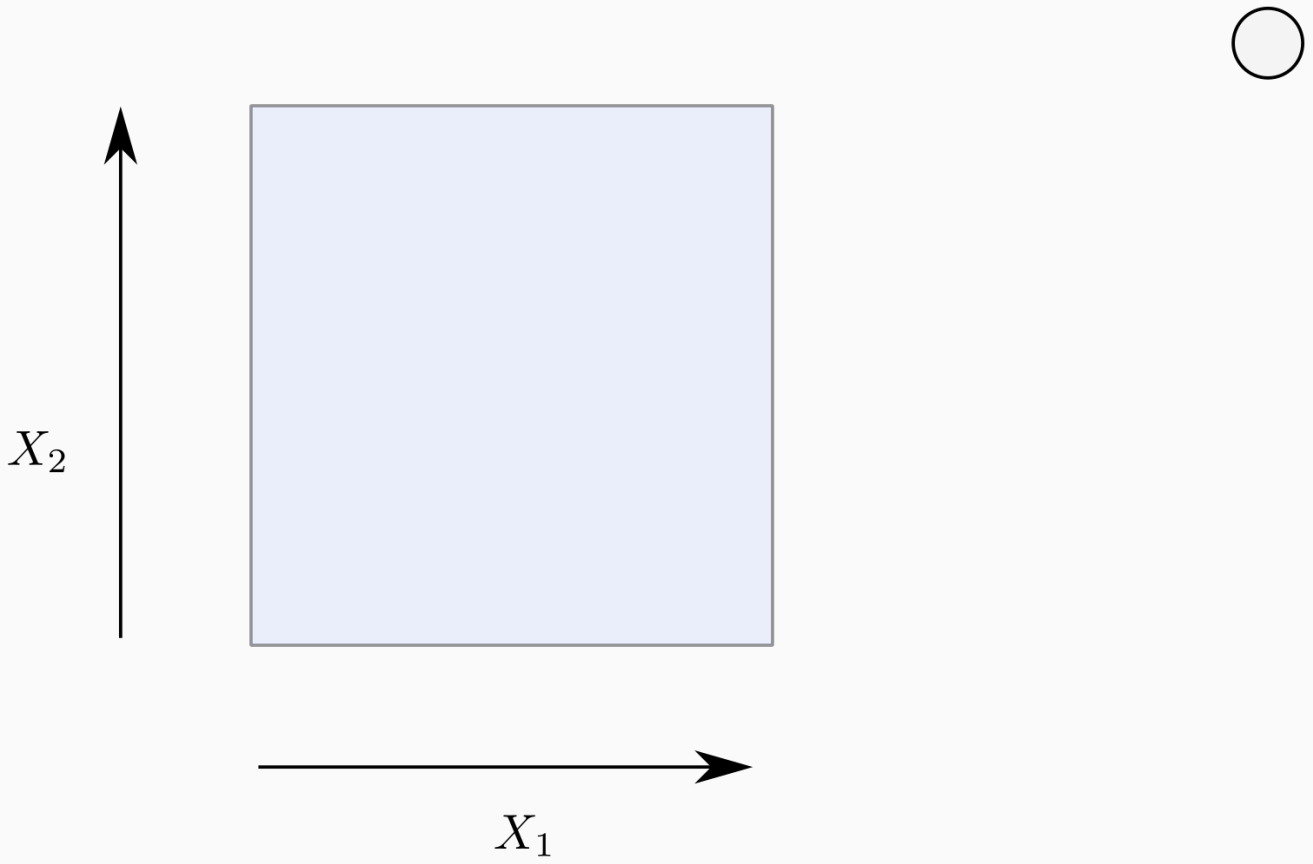
features **x**: weight, diameter, color, acid, ...



Probabilities express uncertainty!

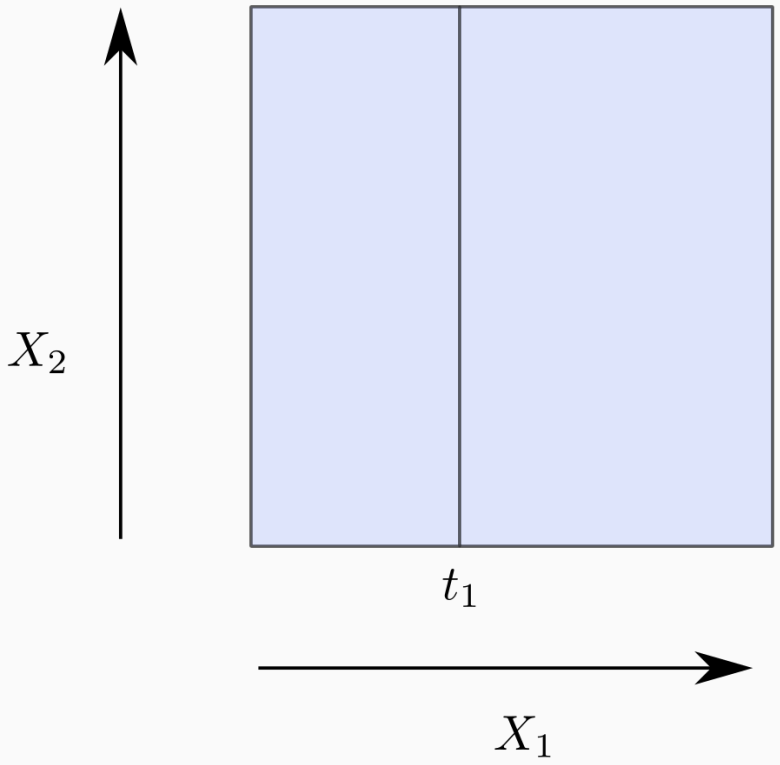
Partition of the Input Space

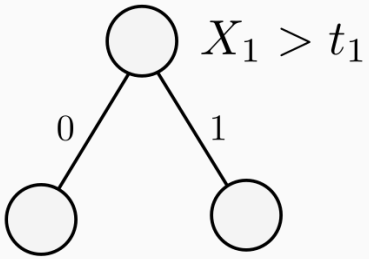
Decision trees represent a partition of the input space:



Partition of the Input Space

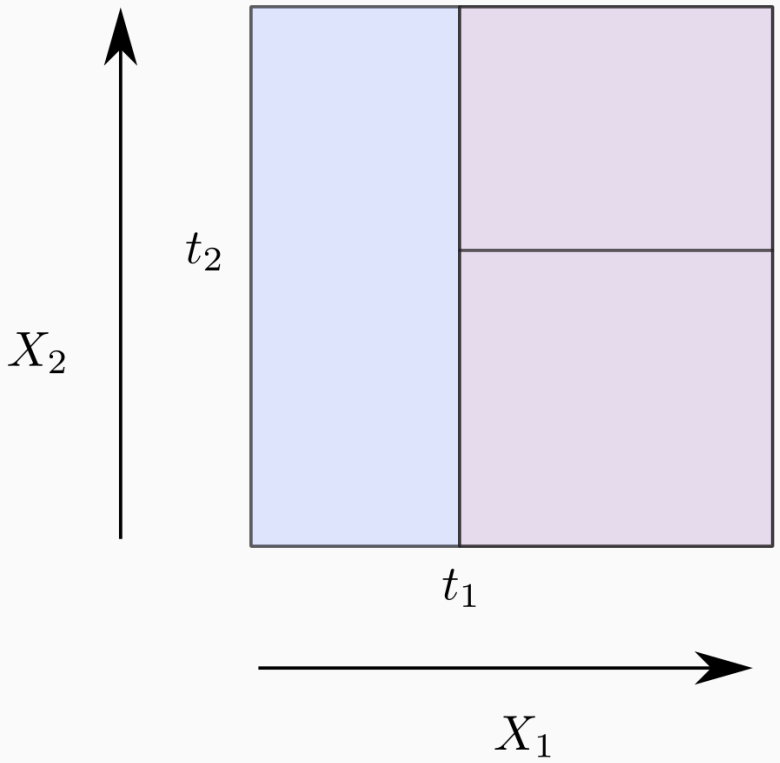
Decision trees represent a partition of the input space:

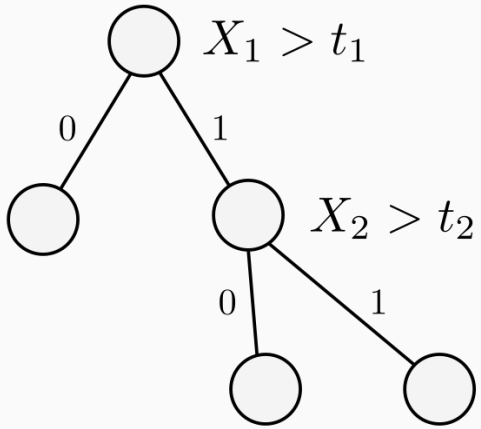




Partition of the Input Space

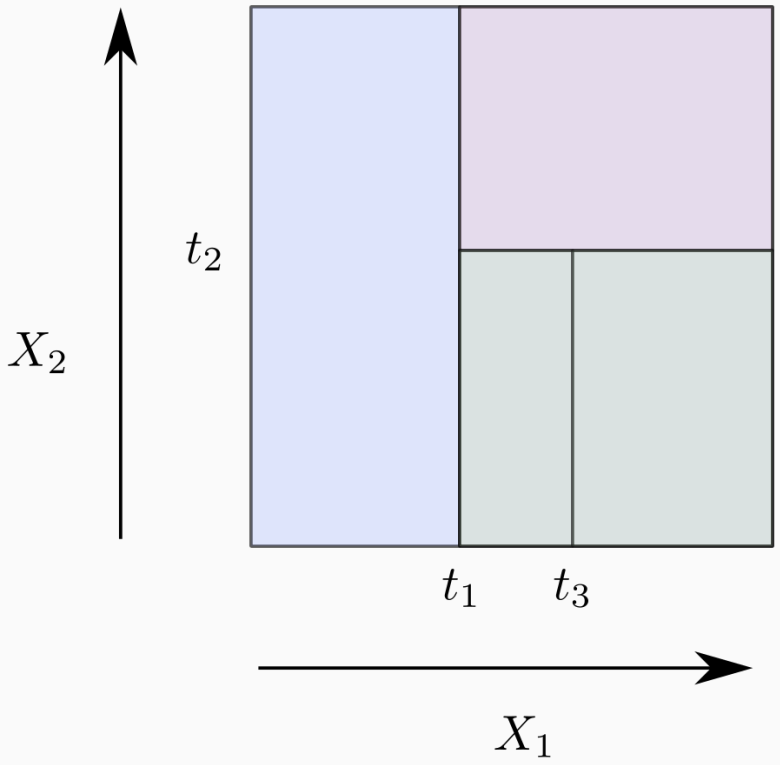
Decision trees represent a partition of the input space:

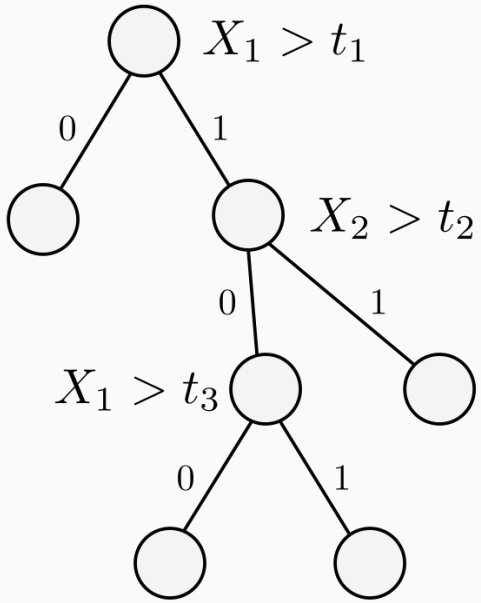




Partition of the Input Space

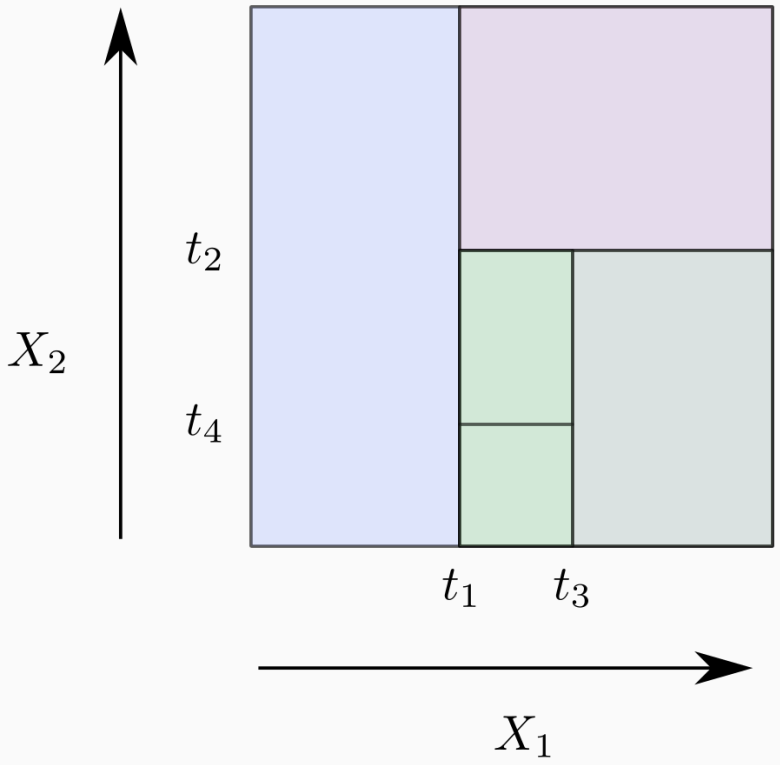
Decision trees represent a partition of the input space:

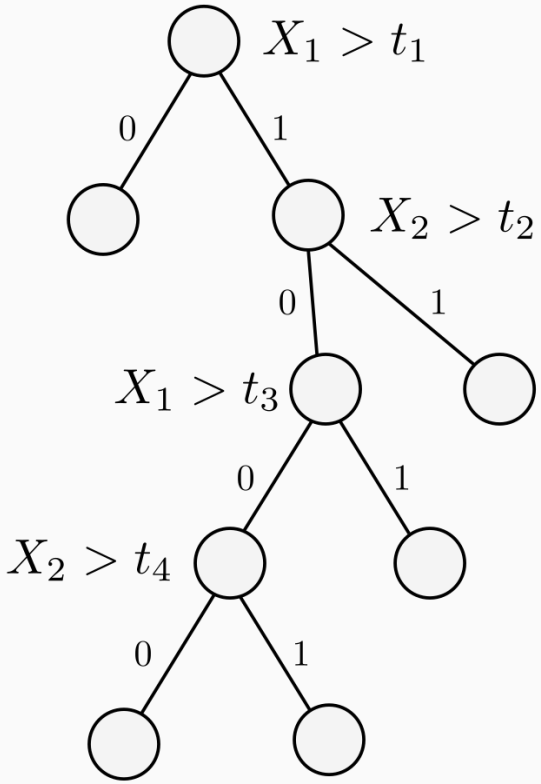




Partition of the Input Space

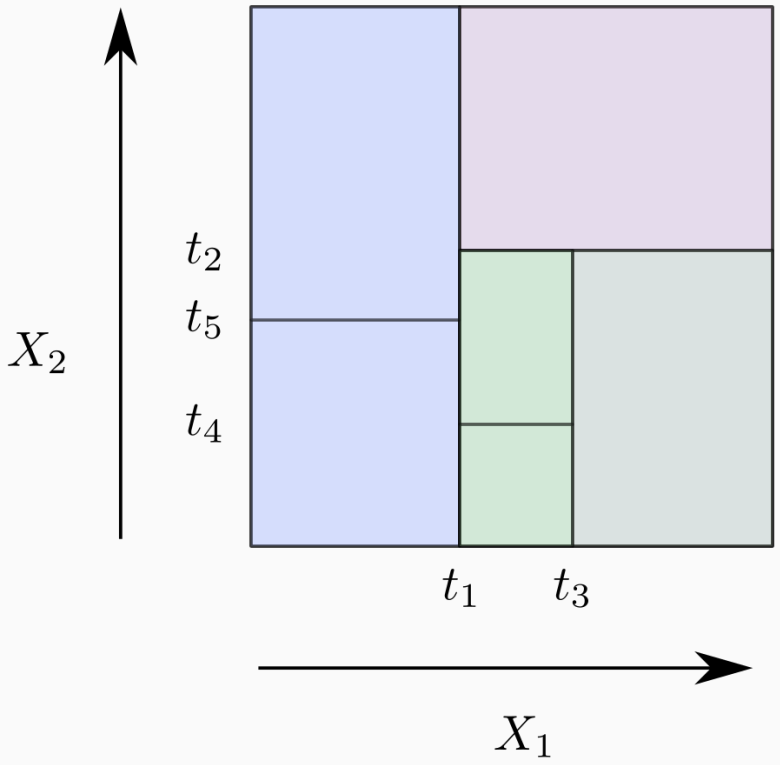
Decision trees represent a partition of the input space:

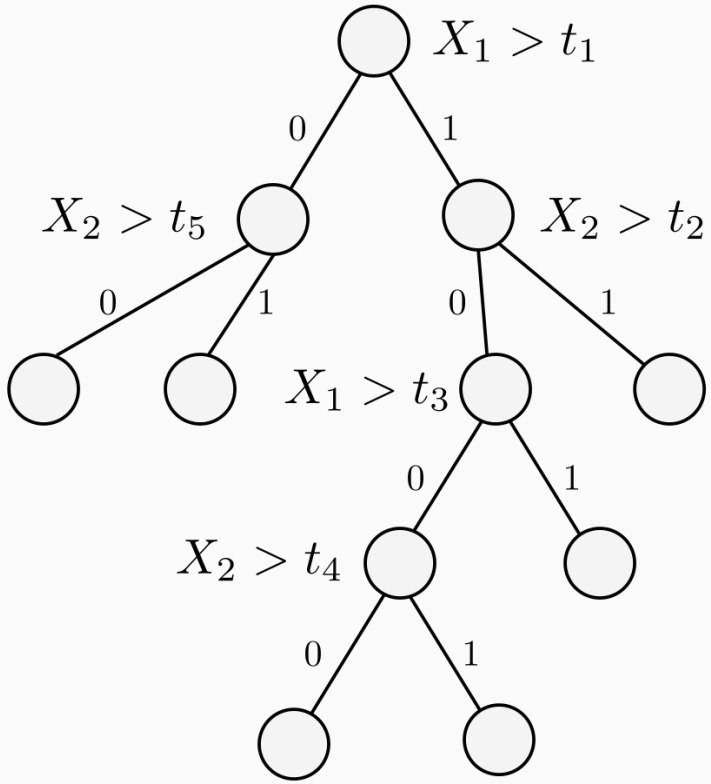




Partition of the Input Space

Decision trees represent a partition of the input space:





Learning Algorithms for Decision Trees

- **CART** (Classification and Regression Tree) [Breiman'84]
- ID3 [Quinlan'86]
- C4.5 [Quinlan'93]
- ...

CART

input: Data $(\boldsymbol{x}_1, y_1), (\boldsymbol{x}_2, y_2), \ldots, (\boldsymbol{x}_N, y_N)$
result: Decision Tree
Initialize single leaf L, assign all data points $\boldsymbol{x}^{(i)}$ to L, call **split**(L)

split(L):
if *stopping criterion is true for* L **then**
    Learn $p(y)$ from class proportions of samples assigned to L
    return
end
for *considered variables/decisions* X_d, f **do**
    compute **cost**(X_d, f)
end
Let X_{min}, f_{min} be the decision with minimal cost
Apply $f_{min}(X_{min})$ to L, yielding new leaves L_0 and L_1
Assign $\boldsymbol{x}^{(i)}$ with $f(\boldsymbol{x}^{(i)}) = \mathit{false}$ to L_0, call **split**(L_0)
Assign $\boldsymbol{x}^{(i)}$ with $f(\boldsymbol{x}^{(i)}) = \mathit{true}$ to L_1, call **split**(L_1)

CART

input: Data $(\boldsymbol{x}_1, y_1), (\boldsymbol{x}_2, y_2), \ldots, (\boldsymbol{x}_N, y_N)$
result: Decision Tree
Initialize single leaf L, assign all data points $\boldsymbol{x}^{(i)}$ to L, call **split**(L)

split(L):
if stopping criterion *is true for* L **then**
    Learn $p(y)$ from class proportions of samples assigned to L
    return
end
for considered variables/decisions X_d, f **do**
    compute **cost**(X_d, f)
end
Let X_{min}, f_{min} be the decision with minimal **cost**
Apply $f_{min}(X_{min})$ to L, yielding new leaves L_0 and L_1
Assign $\boldsymbol{x}^{(i)}$ with $f(\boldsymbol{x}^{(i)}) = \mathit{false}$ to L_0, call **split**(L_0)
Assign $\boldsymbol{x}^{(i)}$ with $f(\boldsymbol{x}^{(i)}) = \mathit{true}$ to L_1, call **split**(L_1)

Possible Decisions

input : Data $(\boldsymbol{x}_1, y_1), (\boldsymbol{x}_2, y_2), \ldots, (\boldsymbol{x}_N, y_N)$
Result: Decision Tree

Initialize single leaf L, assign all data points $\boldsymbol{x}^{(i)}$ to L, call **split**(L)

split(L):
if *stopping criterion is true for* L **then**
    Learn $p(Y)$ from class proportions of samples assigned to L
    return
end
for considered variables/decisions X_d, f **do**
    compute **cost**(X_d, f)
end
Let X_{min}, f_{min} be the decision with minimal cost
Apply X_{min}, f_{min} to L, yielding new leaves L_0 and L_1
Assign $\boldsymbol{x}^{(i)}$ with $f(\boldsymbol{x}^{(i)}) = \mathit{false}$ to L_0, call **split**(L_0)
Assign $\boldsymbol{x}^{(i)}$ with $f(\boldsymbol{x}^{(i)}) = \mathit{true}$ to L_1, call **split**(L_1)

Which Decisions to Consider?

Continuous Feature X_d

- **thresholds** $X_d > t$
- sort data values for X_d
- consider thresholds **halfway between consecutive values**
- only finitely many decisions to consider
- e.g., for values $X_d \in [0.1,\ 1.3,\ 3.1415]$, we consider thresholds $\frac{0.1+1.3}{2} = 0.7$ and $\frac{1.3+3.1415}{2} = 2.22$

Discrete Feature X_d

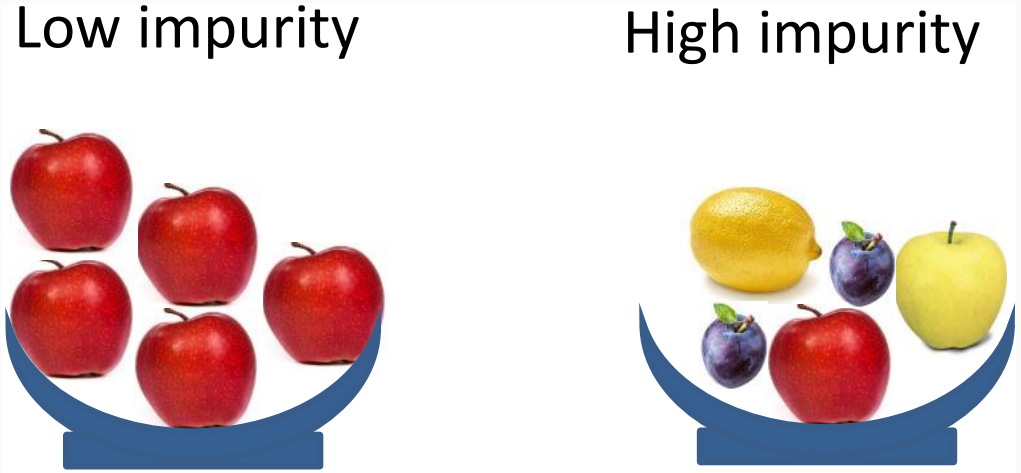
- **one-vs-all** $\{1, 2, 3, 4\} \to \{1, 2, 4\}, \{3\}$
- **complete split** (not a binary tree)
  $\{1, 2, 3, 4\} \to \{1\}, \{2\}, \{3\}, \{4\}$

Cost

input : Data $(\boldsymbol{x}_1, y_1), (\boldsymbol{x}_2, y_2), \ldots, (\boldsymbol{x}_N, y_N)$
Result: Decision Tree
Initialize single leaf L, assign all data points $\boldsymbol{x}^{(i)}$ to L, call **split**(L)

split(L):
if *stopping criterion is true for* L **then**
    Learn $p(Y)$ from class proportions of samples assigned to L
    return
end
for *considered variables/decisions* X_d, f **do**
    compute **cost**(X_d, f)
end
Let X_{min}, f_{min} be the decision with minimal **cost**
Apply X_{min}, f_{min} to L, yielding new leaves L_0 and L_1
Assign $\boldsymbol{x}^{(i)}$ with $f(\boldsymbol{x}^{(i)}) = \mathit{false}$ to L_0, call **split**(L_0)
Assign $\boldsymbol{x}^{(i)}$ with $f(\boldsymbol{x}^{(i)}) = \mathit{true}$ to L_1, call **split**(L_1)

Cost



- "pure leaves are good"
- let N be the number of data points at leaf L
- for a particular **candidate decision** let N_0, N_1 be the number of data points at new leaves L_0 and L_1, respectively
- $\mathit{cost} = \frac{N_0}{N}\mathit{impurity}(L_0) + \frac{N_1}{N}\mathit{impurity}(L_1)$
- note that $\frac{N_0}{N} + \frac{N_1}{N} = 1$ (weighted average)

Impurity Measures

Let $p(y)$ be the empirical frequencies of class labels, e.g.



$$p(\text{plum}) = \frac{5}{8} = 0.625$$
$$p(\text{lemon}) = \frac{3}{8} = 0.375$$

- **Gini Impurity**

$$G = 1 - \sum_y p(y)^2$$

- **Entropy**

$$E = -\sum_y p(y) \log(p(y))$$

- **Self-Classification Error**

$$C = 1 - \max_y p(y)$$

Impurity measures (here for binary classification), as a function of $p(y = 0)$ (and $p(y = 1)$ due to symmetry). Entropy has been scaled to pass through $(0.5, 0.5)$.

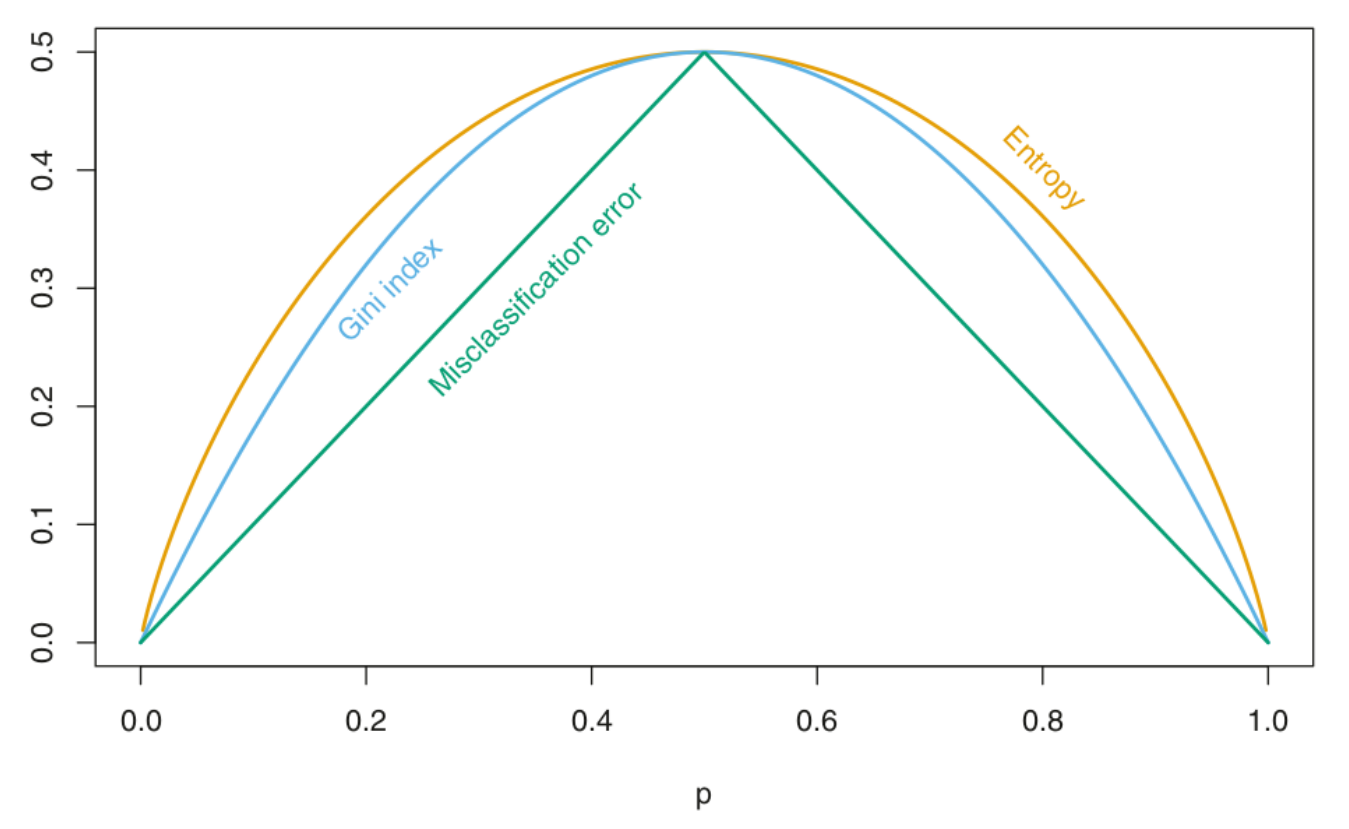


Image: Trevor Hastie et al., *Elements of Statistical Learning*, 2009.

Cost

- let N be the number of data points at leaf L
- let N_0, N_1 be the number of data points at new leaves L_0 and L_1, respectively.
- $\mathit{cost} = \frac{N_0}{N} \mathit{impurity}(L_0) + \frac{N_1}{N} \mathit{impurity}(L_1)$
- where impurity might be
  - **Gini Impurity**

$$G = 1 - \sum_y p(y)^2$$

  - **Entropy**

$$E = -\sum_y p(y) \log(p(y))$$

  - **Self-Classification Error**

$$C = 1 - \max_y p(y)$$

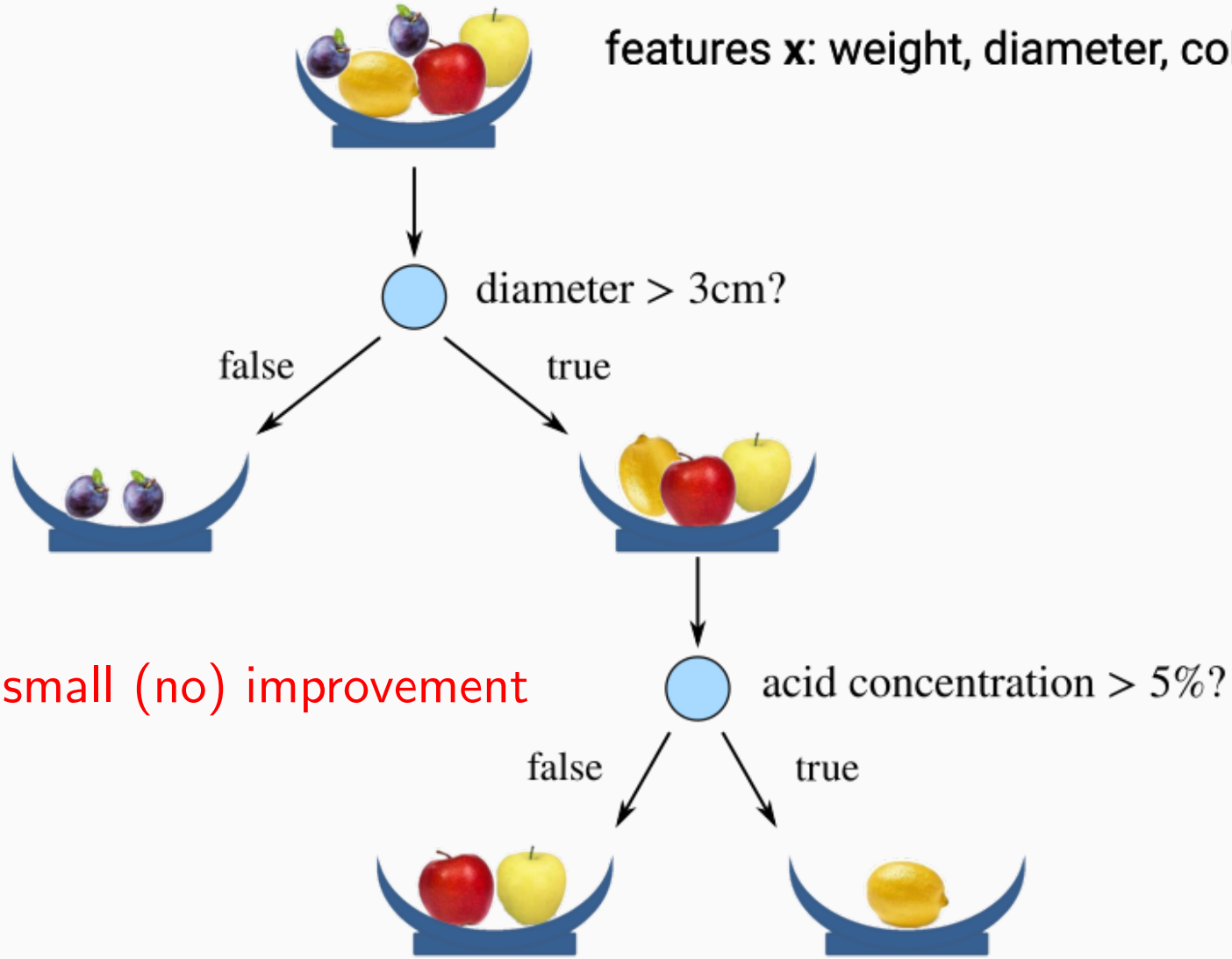
Stopping Criterion

input : Data $(\boldsymbol{x}_1, y_1), (\boldsymbol{x}_2, y_2), \ldots, (\boldsymbol{x}_N, y_N)$
Result: Decision Tree

Initialize single leaf L, assign all data points $\boldsymbol{x}^{(i)}$ to L, call **split**(L)

split(L):
if **stopping criterion** *is true for* L **then**
    Learn $p(Y)$ from class proportions of samples assigned to L
    return
end
for *considered variables/decisions* X_d, f **do**
    compute **cost**(X_d, f)
end
Let X_{min}, f_{min} be the decision with minimal cost
Apply X_{min}, f_{min} to L, yielding new leaves L_0 and L_1
Assign $\boldsymbol{x}^{(i)}$ with $f(\boldsymbol{x}^{(i)}) = \mathit{false}$ to L_0, call **split**(L_0)
Assign $\boldsymbol{x}^{(i)}$ with $f(\boldsymbol{x}^{(i)}) = \mathit{true}$ to L_1, call **split**(L_1)

Stopping Criterion



too small (no) improvement

too small (no) improvement

too few data points

Stopping Criterion and Hyperparameters

- threshold determining when improvement of cost is too small
- another threshold for determining when there are too few data points
- furthermore, one introduce a maximal tree depth
- these thresholds are **hyper-parameters** of CART

Random Forests

- decision trees: highly **interpretable** models, but don't perform too well in practice
- **random forests**: **ensemble** of decision trees
- output of random forest is computed by **aggregating** the outputs of the individual decision trees
  - majority vote for classification
  - averaging for regression
- a theoretical underpinning for ensembles is the **bias-variance** trade-off

Bias-Variance Trade-Off

- let $\mathcal{H}$ be our considered **hypothesis class**
- recall from Lecture 6 that the ideal goal of supervised learning is to find an optimal predictor f^* minimizing the **true loss**:

$$f^* = \arg\min_{f \in \mathcal{H}} \mathcal{L}_{true}(f) = \mathbb{E}_{p_{true}}[\ell(f(\boldsymbol{x}), y)]$$

- however, we only have a finite training set of N samples drawn from p_{true}
- at best, we can find a minimizer f^e of the **empirical loss**:

$$f^e = \arg\min_{f \in \mathcal{H}} \mathcal{L}_{empirical}(f) = \frac{1}{N} \sum_{i=1}^{N} \ell(f(\boldsymbol{x}^{(i)}, y^{(i)}))$$

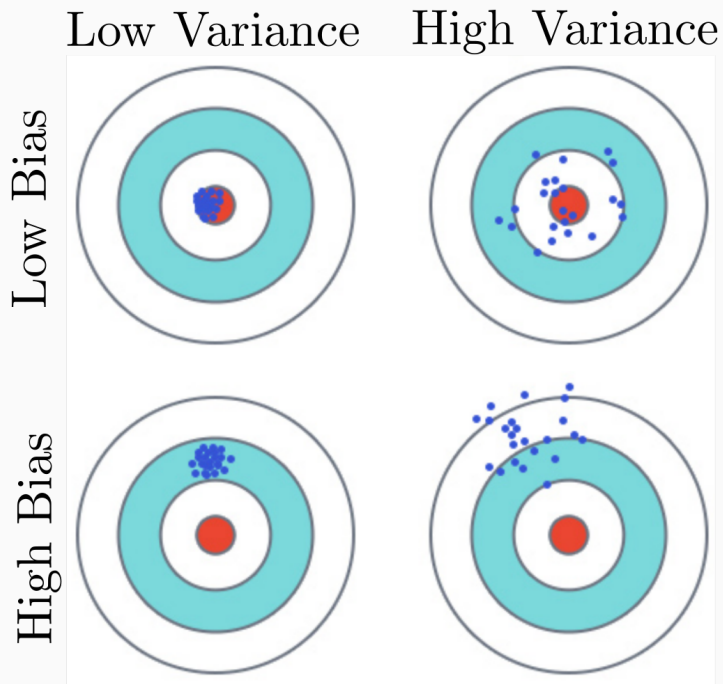
The Bias-Variance Trade-Off

- figure that we can repeatedly draw training sets of size N from p_{true}:
  - the training set is random
  - thus, the learned model f^{emp} is random
  - thus, $\mathcal{L}_{true}(f^{emp})$ is also random
  - thus, the **excess loss** $\delta = \mathcal{L}_{true}(f^{emp}) - \mathcal{L}_{true}(f^*)$ is also random
- $\mathbb{E}[\delta]$ is denoted as **bias**, measuring how far $\mathcal{L}_{true}(f^{emp})$ is from $\mathcal{L}_{true}(f^*)$ on average
- the **variance** $\mathbf{var}[\delta] = \mathbb{E}\left[(\delta - \mathbb{E}[\delta])^2\right]$ measures how much the $\mathcal{L}_{true}(f^{emp})$ varies across repeated draws of training sets

The Bias-Variance Trade-Off cont'd

Bias and variance depend on the model complexity:

- simple models tend to have **high bias** and **low variance** $\Rightarrow$ **underfitting**
- expressive, flexible models tend to have **low bias** and **high variance** $\Rightarrow$ **overfitting**



Why Ensembles?

- individual models (like decision trees) easily overfit, i.e. they have low bias but high variance
- **idea:** instead of training one model, train K models and aggregate them
- we can expect that $\textbf{var}(\text{ensemble}) \approx \frac{\textbf{var}(\text{single model})}{K}$
- what is the problem with this idea?

- we have only one dataset—hence use **bootstrapping** to generate synthetic copies of the training data
- in statistics, bootstrapping is a **resampling method** to produces uncertainty estimates
- given N samples, generate K new datasets of size N by **sampling with replacement**
- these new datasets are called **bootstraps**
- for large N, each bootstrap contains only $1 - \frac{1}{e} \approx 63.21\%$ of the original samples

Original Data	x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8	x_9	x_{10}
Boostrap 1	x_7	x_8	x_{10}	x_8	x_2	x_5	x_{10}	x_{10}	x_5	x_9
Boostrap 2	x_1	x_1	x_9	x_1	x_2	x_3	x_2	x_7	x_3	x_2
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
Boostrap K	x_1	x_8	x_5	x_{10}	x_5	x_5	x_9	x_6	x_3	x_7

- **bagging** with Decision Trees
- **bagging = bootstrapping and aggregating**
- generate K bootstraps, and learn a decision tree on each of them
- additionally, randomize CART learning (to get more diverse trees): rather than considering all variables for a new split, restrict to m randomly chosen variables, $m < D$
- typical values, $K = 100, 200, 500$, $m \approx \frac{D}{3}$
- run-time: K times the run-time of decision trees

Decision Tree vs. Random Forest

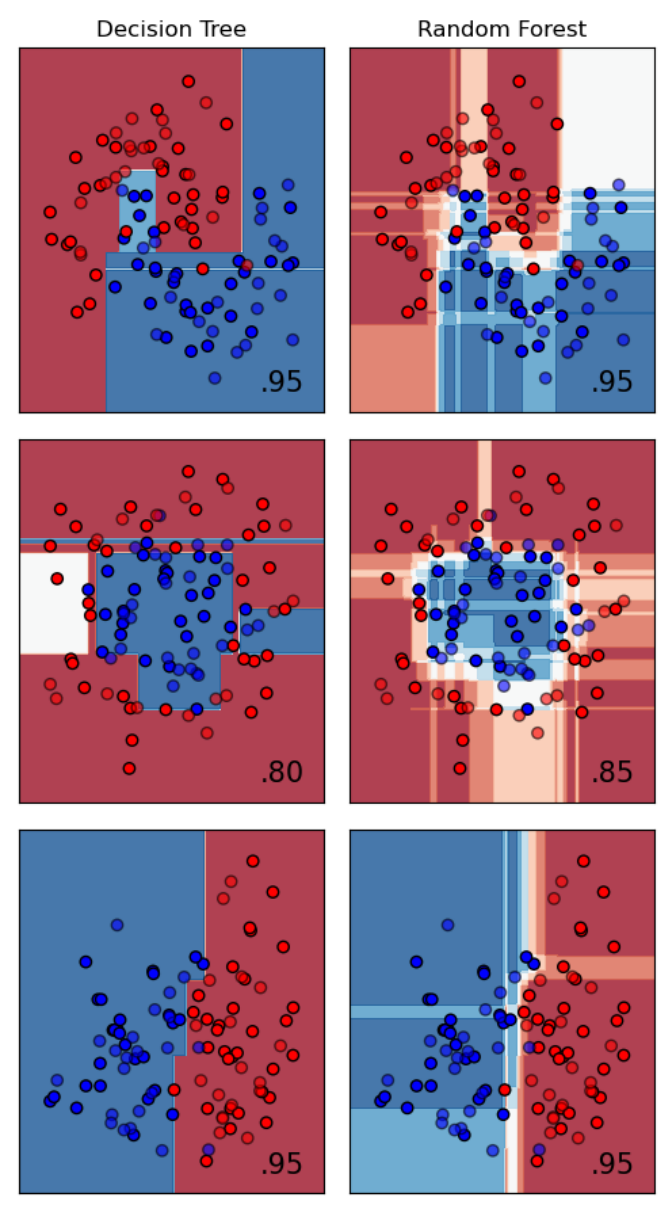
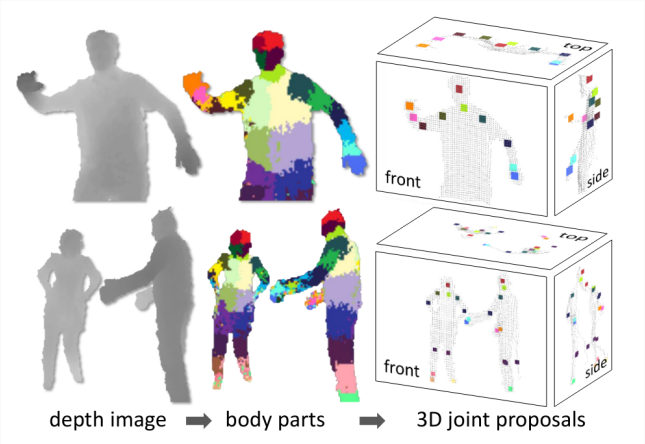


Image: https://scikit-learn.org/

Random Forests in Practice

- Kaggle reported in 2019 that decision trees and random forests are the most widely used tool in data mining and applied machine learning, after logistic regression
- Fernánandez-Delgado et al. (2014) random forests perform best among a wide range of algorithms on **tabular data**
- prominent example: body-part detection for skeleton tacking (Kinect 2)



Shotton et al., Real-Time Human Pose Recognition in Parts from Single Depth Images, 2011

Summary

- **decision trees**: top-down decision diagram
- decision nodes split input space recursively
- leaves contain localized predictors
- CART algorithm: recursively find optimal decision (split) by minimizing impurity
- **random forest**: randomized ensemble of decision trees
- **bagging**: bootstrapping and aggregating